

6. Linear Model Selection and Regularization

6.1 Subset selection

6.2 Shrinkage methods by regularization

6.3 High dimensional data analysis

6. Linear Model Selection and Regularization

KNN regression for best subset selection

```
##### KNN regression for best subset selection
import numpy as np
import pandas as pd
from itertools import combinations
from sklearn.datasets import load_boston
from sklearn.model_selection import cross_val_score, KFold
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from joblib import Parallel, delayed
import warnings
warnings.filterwarnings('ignore') # Suppress warnings for cleaner output

def evaluate_subset(subset, X, y, model, scoring, cv_folds, random_state):
    X_subset = X[list(subset)]
    # Perform cross-validation
    scores = cross_val_score(model, X_subset, y, cv=KFold(n_splits=cv_folds, shuffle=True, random_state=random_state), scoring=scoring)
    mean_score = np.mean(scores)
    return (subset, -mean_score)

def best_subset_selection_parallel(X, y, model, scoring='neg_mean_squared_error', cv_folds=5, n_jobs=-1, random_state=42):
    feature_names = X.columns
    n_features = len(feature_names)
    results = []
    for k in range(1, n_features + 1):
        print(f"Evaluating {k}-feature subsets...")
        subsets = list(combinations(feature_names, k))
        subset_scores = Parallel(n_jobs=n_jobs)(delayed(evaluate_subset)(subset, X, y, model, scoring, cv_folds, random_state) for subset in subsets)
        results.extend(subset_scores)

    return results

def main():
    # Load the Dataset
    Credit = pd.read_csv('Credit.csv')
    Credit = Credit.drop('ID', axis=1)
    print(Credit.head(10))
    [num_samples, num_predictors] = Credit.shape
    Credit['Gender'] = Credit['Gender'].map({'Male': 1, 'Female': 0})
    Credit['Student'] = Credit['Student'].map({'Yes': 1, 'No': 0})
    Credit['Married'] = Credit['Married'].map({'Yes': 1, 'No': 0})
    Credit['Ethnicity'] = Credit['Ethnicity'].map({'Asian': 2, 'Caucasian': 1, 'African American': 0})
    X = Credit.drop('Balance', axis=1)
    y = Credit['Balance']
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

    knn = KNeighborsRegressor(n_neighbors=10)
    print("Starting Best Subset Selection with KNN Regression (Parallel)...")
    results = best_subset_selection_parallel(X_scaled, y, model=knn, scoring='neg_mean_squared_error', cv_folds=5, n_jobs=-1, random_state=42)
    print("Best Subset Selection Completed.")
    results_df = pd.DataFrame(results, columns=['Features', 'CV_Score'])
    results_df = results_df.sort_values(by='CV_Score', ascending=True).reset_index(drop=True)

    print("\nTop 20 Best Feature Subsets:")
    print(results_df.head(20))
    best_features = results_df.iloc[0]['Features']
    print(f"\nBest feature subset: {best_features}")
    X_best = X_scaled[list(best_features)]
    final_scores = cross_val_score(knn, X_best, y, cv=KFold(n_splits=5, shuffle=True, random_state=42), scoring='neg_mean_squared_error')
    final_rmse = np.sqrt(-final_scores)

    print(f"\nCross-Validated RMSE of the best model: {final_rmse}")
    print(f"Average RMSE: {final_rmse.mean():.4f}")
    print(f"Std of RMSE: {final_rmse.std():.4f}")

if __name__ == "__main__":
    main()
```

6. Linear Model Selection and Regularization

KNN regression for best subset selection

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	\
0	14.891	3606	283	2	34	11	Male	No	Yes	
1	106.025	6645	483	3	82	15	Female	Yes	Yes	
2	104.593	7075	514	4	71	11	Male	No	No	
3	148.924	9504	681	3	36	11	Female	No	No	
4	55.882	4897	357	2	68	16	Male	No	Yes	
5	80.180	8047	569	4	77	10	Male	No	No	
6	20.996	3388	259	2	37	12	Female	No	No	
7	71.408	7114	512	2	87	9	Male	No	No	
8	15.125	3300	266	5	66	13	Female	No	No	
9	71.061	6819	491	3	41	19	Female	Yes	Yes	

	Ethnicity	Balance
0	Caucasian	333
1	Asian	903
2	Asian	580
3	Asian	964
4	Caucasian	331
5	Caucasian	1151
6	African American	203
7	Asian	872
8	Caucasian	279
9	African American	1350

Starting Best Subset Selection with KNN Regression (Parallel)...

Evaluating 1-feature subsets...

Evaluating 2-feature subsets...

Evaluating 3-feature subsets...

Evaluating 4-feature subsets...

Evaluating 5-feature subsets...

Evaluating 6-feature subsets...

Evaluating 7-feature subsets...

Evaluating 8-feature subsets...

Evaluating 9-feature subsets...

Evaluating 10-feature subsets...

Best Subset Selection Completed.

6. Linear Model Selection and Regularization

KNN regression for best subset selection

Top 20 Best Feature Subsets:

	Features	CV_Score
0	(Income, Limit, Rating, Student)	12602.620575
1	(Income, Limit, Student)	14620.083475
2	(Income, Rating, Student)	16046.909125
3	(Income, Limit, Rating, Student, Ethnicity)	20242.972425
4	(Income, Limit, Rating, Cards, Student)	22041.113750
5	(Income, Limit, Rating, Gender, Student)	22126.740425
6	(Income, Limit, Rating, Age, Student)	24270.743075
7	(Income, Limit, Rating, Education, Student)	24404.215800
8	(Income, Limit, Rating, Student, Married)	25055.440775
9	(Income, Limit, Rating)	26080.494850
10	(Income, Limit, Student, Ethnicity)	27375.569975
11	(Income, Limit, Cards, Student)	27904.832900
12	(Income, Rating)	27983.625475
13	(Income, Rating, Student, Ethnicity)	28141.017050
14	(Income, Limit)	28662.090225
15	(Income, Limit, Age, Student)	30202.329950
16	(Income, Rating, Cards, Student)	30208.340050
17	(Income, Rating, Age, Student)	30232.713850
18	(Income, Limit, Rating, Gender, Student, Married)	30497.192750
19	(Income, Limit, Gender, Student)	30564.526075

Best feature subset: ('Income', 'Limit', 'Rating', 'Student')

Cross-Validated RMSE of the best model: [105.40098849 98.76796609 81.93983006 145.26987558 119.71278493]

Average RMSE: 110.2183

Std of RMSE: 21.3202

6. Linear Model Selection and Regularization

KNN regression for best subset selection with MANY variables

```
##### KNN regression for best subset selection with noisy variables
import numpy as np
import pandas as pd
from itertools import combinations
from sklearn.datasets import load_boston
from sklearn.model_selection import cross_val_score, KFold
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from joblib import Parallel, delayed
import warnings
warnings.filterwarnings('ignore') # Suppress warnings for cleaner output

def evaluate_subset(subset, X, y, model, scoring, cv_folds, random_state):
    X_subset = X[list(subset)]
    # Perform cross-validation
    scores = cross_val_score(model, X_subset, y, cv=KFold(n_splits=cv_folds, shuffle=True, random_state=random_state), scoring=scoring)
    mean_score = np.mean(scores)
    return (subset, -mean_score)

def best_subset_selection_parallel(X, y, model, scoring='neg_mean_squared_error', cv_folds=5, n_jobs=-1, random_state=42):
    feature_names = X.columns
    #n_features = len(feature_names) ##### maximum model size
    n_features=3
    results = []
    for k in range(1, n_features + 1):
        print(f"Evaluating {k}-feature subsets...")
        subsets = list(combinations(feature_names, k))
        subset_scores = Parallel(n_jobs=n_jobs)(delayed(evaluate_subset)(subset, X, y, model, scoring, cv_folds, random_state) for subset in subsets)
        results.extend(subset_scores)

    return results

def main():
    # Load the Dataset
    Credit = pd.read_csv('Credit.csv')
    Credit = Credit.drop('ID', axis=1)
    [num_samples, num_predictors] = Credit.shape
    Credit['Gender'] = Credit['Gender'].map({'Male': 1, 'Female': 0})
    Credit['Student'] = Credit['Student'].map({'Yes': 1, 'No': 0})
    Credit['Married'] = Credit['Married'].map({'Yes': 1, 'No': 0})
    Credit['Ethnicity'] = Credit['Ethnicity'].map({'Asian': 2, 'Caucasian': 1, 'African American': 0})
    ##### create noisy columns
    column_names = [f'Noisy{i}' for i in range(1, 20 + 1)]
    data = {col: np.random.normal(loc=0, scale=1, size=400) for col in column_names}
    Credit = pd.concat([Credit, pd.DataFrame(data)], axis=1, sort=False)
    print(Credit.head(4))

    X = Credit.drop('Balance', axis=1)
    y = Credit['Balance']
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

    knn = KNeighborsRegressor(n_neighbors=10)
    print("Starting Best Subset Selection with KNN Regression (Parallel)...")
    results = best_subset_selection_parallel(X_scaled, y, model=knn, scoring='neg_mean_squared_error', cv_folds=5, n_jobs=-1, random_state=42)
    print("Best Subset Selection Completed.")
    results_df = pd.DataFrame(results, columns=['Features', 'CV_Score'])
    results_df = results_df.sort_values(by='CV_Score', ascending=True).reset_index(drop=True)

    print("\nTop 20 Best Feature Subsets:")
    print(results_df.head(20))
    best_features = results_df.iloc[0]['Features']
    print(f"\nBest feature subset: {best_features}")
    X_best = X_scaled[list(best_features)]
    final_scores = cross_val_score(knn, X_best, y, cv=KFold(n_splits=5, shuffle=True, random_state=42), scoring='neg_mean_squared_error')
    final_rmse = np.sqrt(-final_scores)

    print(f"\nCross-Validated RMSE of the best model: {final_rmse}")
    print(f"Average RMSE: {final_rmse.mean():.4f}")
    print(f"Std of RMSE: {final_rmse.std():.4f}")

if __name__ == "__main__":
    main()
```

6. Linear Model Selection and Regularization

KNN regression for best subset selection with MANY variables

```

Income  Limit  Rating  Cards  Age  Education  Gender  Student  Married  \
0  14.891  3606   283    2   34         11      1        0        1
1  106.025 6645   483    3   82         15      0        1        1
2  104.593 7075   514    4   71         11      1        0        0
3  148.924 9504   681    3   36         11      0        0        0

Ethnicity  ...  Noisy11  Noisy12  Noisy13  Noisy14  Noisy15  Noisy16  \
0          1  ...  2.546853  2.103351 -0.170147 -0.265479  0.572894 -1.310142
1          2  ... -0.770226 -0.017263  1.093138 -1.510666  1.192295 -0.347671
2          2  ...  0.632699 -0.265577  1.400248  1.505839  0.064735  1.341368
3          2  ...  0.261574  0.342526  0.559173  0.236202  0.588366 -0.213027

Noisy17  Noisy18  Noisy19  Noisy20
0  1.396948 -2.421116  0.236121  0.916006
1 -1.680845  0.973348  0.934896  2.318027
2  0.797826 -0.502083  0.816999 -0.664365
3  1.992218  0.130274 -0.485299  0.435085

[4 rows x 31 columns]
Starting Best Subset Selection with KNN Regression (Parallel)...
Evaluating 1-feature subsets...
Evaluating 2-feature subsets...
Evaluating 3-feature subsets...
Best Subset Selection Completed.

Top 20 Best Feature Subsets:
      Features      CV_Score
0  (Income, Limit, Student)  14620.083475
1  (Income, Rating, Student)  16046.909125
2  (Income, Limit, Rating)   26080.494850
3  (Income, Rating)         27983.625475
4  (Income, Limit)          28662.090225
5  (Income, Rating, Ethnicity) 35392.342675
6  (Income, Rating, Married)  35397.186975
7  (Income, Limit, Married)   36029.581375
8  (Income, Rating, Gender)   36053.680625
9  (Income, Rating, Noisy10)  36189.818225
10 (Income, Rating, Noisy17)  36226.824550
11 (Income, Rating, Noisy18)  36572.149850
12 (Income, Rating, Noisy3)   36654.618975
13 (Income, Limit, Gender)    36676.519200
14 (Limit, Student)          36843.472325
15 (Income, Limit, Noisy17)   36848.178625
16 (Income, Limit, Noisy13)   36860.896650
17 (Income, Rating, Noisy16)  36915.908525
18 (Income, Limit, Ethnicity) 36922.465000
19 (Income, Limit, Noisy9)    36993.944450

Best feature subset: ('Income', 'Limit', 'Student')

Cross-Validated RMSE of the best model: [114.49437159 107.31154819 79.97787272 161.47093663 126.51626131]
Average RMSE: 117.9542
Std of RMSE: 26.5874

```

6.1 Subset Selection

▶ Linear model already addressed in detail in Chapter 3.

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$$

▶ Model assessment: cross-validation (prediction) error in Chapter 5.

▶ *This chapter is about model selection for linear models.*

▶ *The model selection techniques can be extended beyond linear models.*

6.1 Subset Selection

Feature/variable selection

▶ *Not all existing input variables are useful for predicting the output.*

▶ *Keeping redundant inputs in model can lead to poor prediction and poor interpretation.*

We consider two ways of variable/model selection:

- ▶ **Subset selection.**
- ▶ **Shrinkage/regularization:** *constraining some regression parameters to 0.*

6.1 Subset Selection

Feature/variable selection

Model 1

use all p predictors

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \quad \longrightarrow \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Model 2

use only q predictors

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1,q} \\ 1 & x_{21} & x_{22} & \cdots & x_{2,q} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{n,q} \end{pmatrix} \quad \longrightarrow \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

How to compare which model is better?

If indeed only the first q predictors are related to the response, model 2 often works better than model 1 because of the many “noise” variables.

The issue is “how to identify which variables are related, and which are not?”

6.1 Subset Selection

Feature/variable selection

Model 1

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1q} \\ 1 & x_{21} & x_{22} & \cdots & x_{2q} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nq} \end{pmatrix}$$



$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

use first q predictors

Model 2

$$\mathbf{X} = \begin{pmatrix} 1 & x_{12} & x_{13} & \cdots & x_{1,q+1} \\ 1 & x_{22} & x_{23} & \cdots & x_{2,q+1} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n2} & x_{n3} & \cdots & x_{n,q+1} \end{pmatrix}$$



$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

use another q predictors

How to compare which model is better?

6.1 Subset Selection

Best subset selection

- ▶ Exhaust all possible combinations of inputs.
- ▶ With p variables, there are 2^p many distinct combinations.
- ▶ Identify the best model among these models.

Algorithm for best subset selection

exhaustive search

- Step 1** Let $\mathcal{M}_0$ be the null model, $Y = \beta_0 + \varepsilon$. The predictor is the sample mean of response.
- Step 2** For $k = 1, 2, \dots, p$,
 - Fit all $\binom{p}{k} = p!/(k!(n-k)!)$ models that contain exactly k predictors.
 - Pick the best model, that with largest R^2 , among them and call it $\mathcal{M}_k$.
- Step 3** Select a single best model from $\mathcal{M}_0, \dots, \mathcal{M}_p$ by cross validation, AIC, or BIC.

6.1 Subset Selection

Best subset selection

Comments

- ▶ *Step 2 reduces to $p + 1$ models, using training error.*
- ▶ *Step 3 identifies the best model using test error.*
- ▶ *Why using R^2 for step 2: they are of same complexity; the RSS is easy to compute.*
- ▶ *Cannot use training error in Step 3.*

6.1 Subset Selection

Recall: residue

Residue

$$\hat{\epsilon}_i = y_i - \hat{\beta}_0 - \sum_{j=1}^p \hat{\beta}_j x_{ij}$$

Residual Sum of Squares

$$\text{RSS} = \sum_{i=1}^n \hat{\epsilon}_i^2 = \sum_{i=1}^n \left(y_i - \hat{\beta}_0 - \sum_{j=1}^p \hat{\beta}_j x_{ij} \right)^2$$

R-squared

$$R^2 = \frac{\text{SS}_{reg}}{\text{SS}_{total}} = 1 - \frac{\text{SS}_{error}}{\text{SS}_{total}}$$

where $\text{SS}_{error} = \text{RSS} = \sum_{i=1}^n \hat{\epsilon}_i^2$ and $\text{SS}_{total} = \sum_{i=1}^n (y_i - \bar{y})^2$.

6.1 Subset Selection

Example: credit data

For example, the **Credit** data set displayed in Figure 3.6 records **balance** (average credit card debt for a number of individuals) as well as several quantitative predictors: **age**, **cards** (number of credit cards), **education** (years of education), **income** (in thousands of dollars), **limit** (credit limit), and **rating** (credit rating). Each panel of Figure 3.6 is a scatterplot for a pair of variables whose identities are given by the corresponding row and column labels. For example, the scatterplot directly to the right of the word “Balance” depicts **balance** versus **age**, while the plot directly to the right of “Age” corresponds to **age** versus **cards**. In addition to these quantitative variables, we also have four qualitative variables: **gender**, **student** (student status), **status** (marital status), and **ethnicity** (Caucasian, African American or Asian).



Figure 3.6 shown next slide

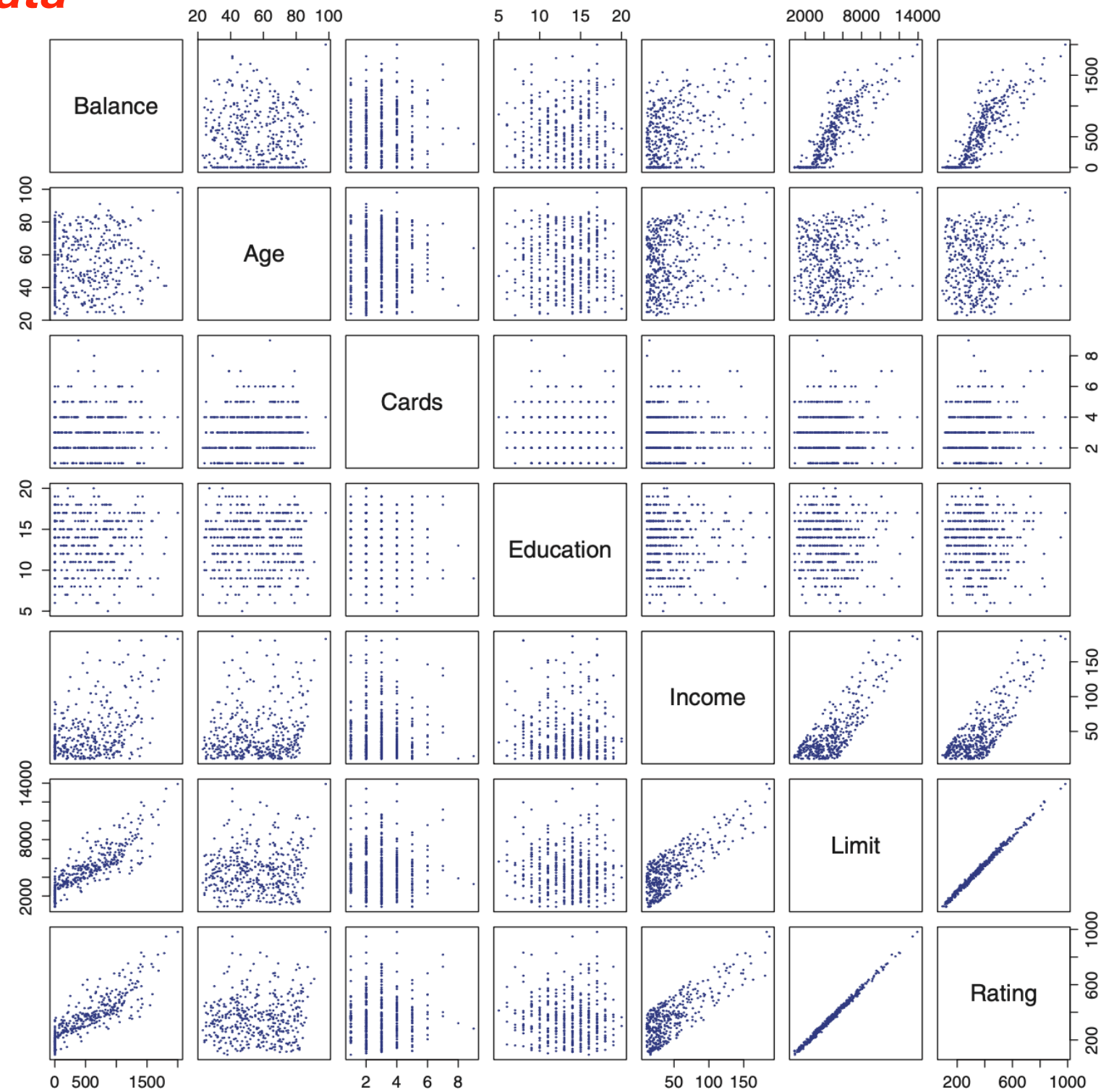
*Example: credit data***6.1 Subset Selection**

FIGURE 3.6. The **Credit** data set contains information about **balance**, **age**, **cards**, **education**, **income**, **limit**, and **rating** for a number of potential customers.

6.1 Subset Selection

Example: credit data

```
> ##### Credit Data
> library(ISLR)
> Credit[1:10,]
```

	ID	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
1	1	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333
2	2	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903
3	3	104.593	7075	514	4	71	11	Male	No	No	Asian	580
4	4	148.924	9504	681	3	36	11	Female	No	No	Asian	964
5	5	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331
6	6	80.180	8047	569	4	77	10	Male	No	No	Caucasian	1151
7	7	20.996	3388	259	2	37	12	Female	No	No	African American	203
8	8	71.408	7114	512	2	87	9	Male	No	No	Asian	872
9	9	15.125	3300	266	5	66	13	Female	No	No	Caucasian	279
10	10	71.061	6819	491	3	41	19	Female	Yes	Yes	African American	1350

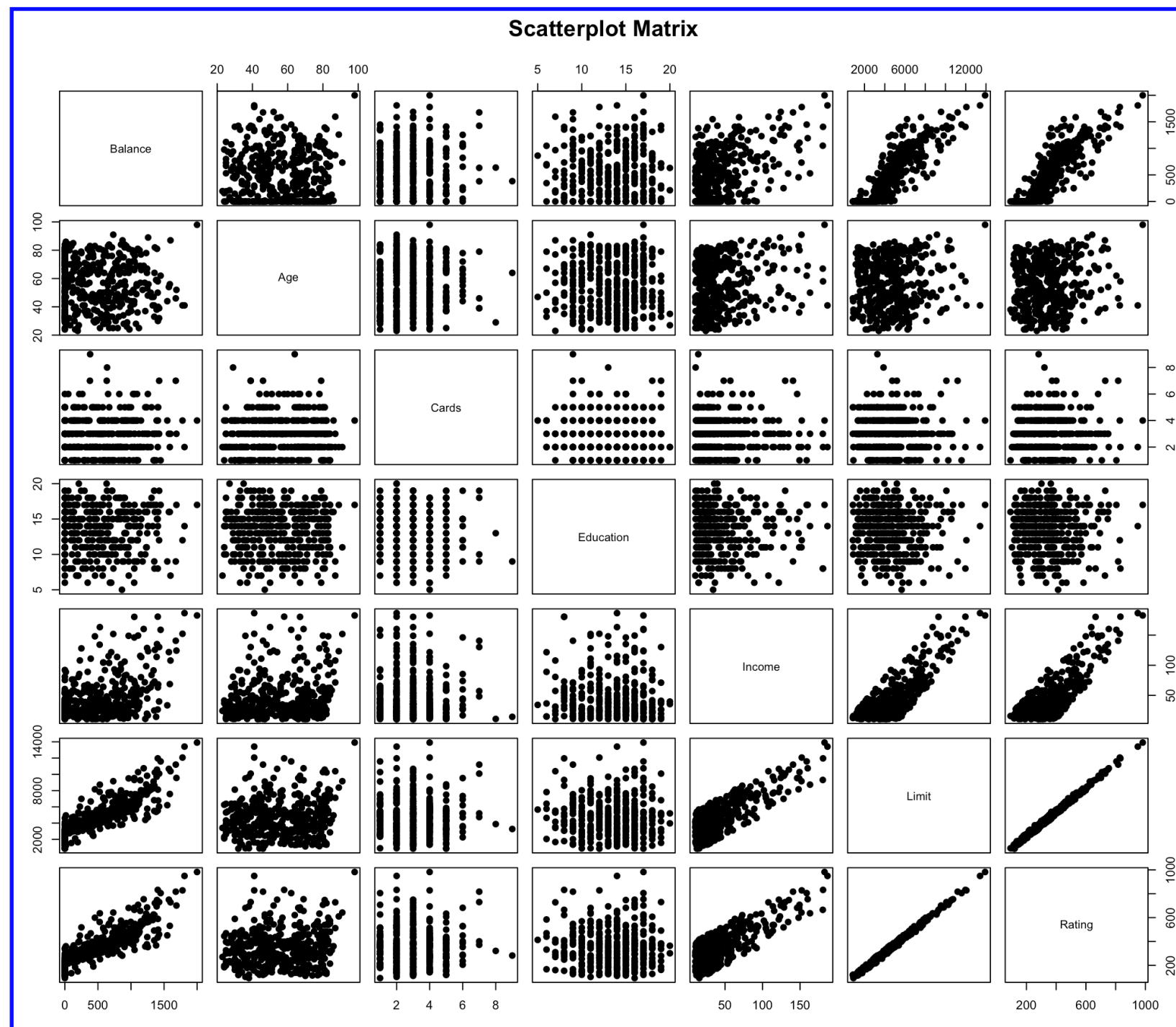
```
> summary(Credit)
```

ID	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity
Min. : 1.0	Min. : 10.35	Min. : 855	Min. : 93.0	Min. : 1.000	Min. : 23.00	Min. : 5.00	Male :193	No :360	No :155	African American: 99
1st Qu.:100.8	1st Qu.: 21.01	1st Qu.: 3088	1st Qu.:247.2	1st Qu.:2.000	1st Qu.:41.75	1st Qu.:11.00	Female:207	Yes: 40	Yes:245	Asian :102
Median :200.5	Median : 33.12	Median : 4622	Median :344.0	Median :3.000	Median :56.00	Median :14.00				Caucasian :199
Mean :200.5	Mean : 45.22	Mean : 4736	Mean :354.9	Mean :2.958	Mean :55.67	Mean :13.45				
3rd Qu.:300.2	3rd Qu.: 57.47	3rd Qu.: 5873	3rd Qu.:437.2	3rd Qu.:4.000	3rd Qu.:70.00	3rd Qu.:16.00				
Max. :400.0	Max. :186.63	Max. :13913	Max. :982.0	Max. :9.000	Max. :98.00	Max. :20.00				

Balance
Min. : 0.00
1st Qu.: 68.75
Median : 459.50
Mean : 520.01
3rd Qu.: 863.00
Max. :1999.00

*Example: credit data***6.1 Subset Selection****ScatterPlot Matrix**

```
pairs(Credit[,c(12,6,5,7,2,3,4)], main="Scatterplot Matrix",pch=19) ## ScatterPlot Matrix
```



6.1 Subset Selection

Example: credit data

```
library(leaps)
regfit.full<-regsubsets(Balance~.,data=Credit,method="exhaustive",nbest=20,nvmax=10)
summary(regfit.full)
```

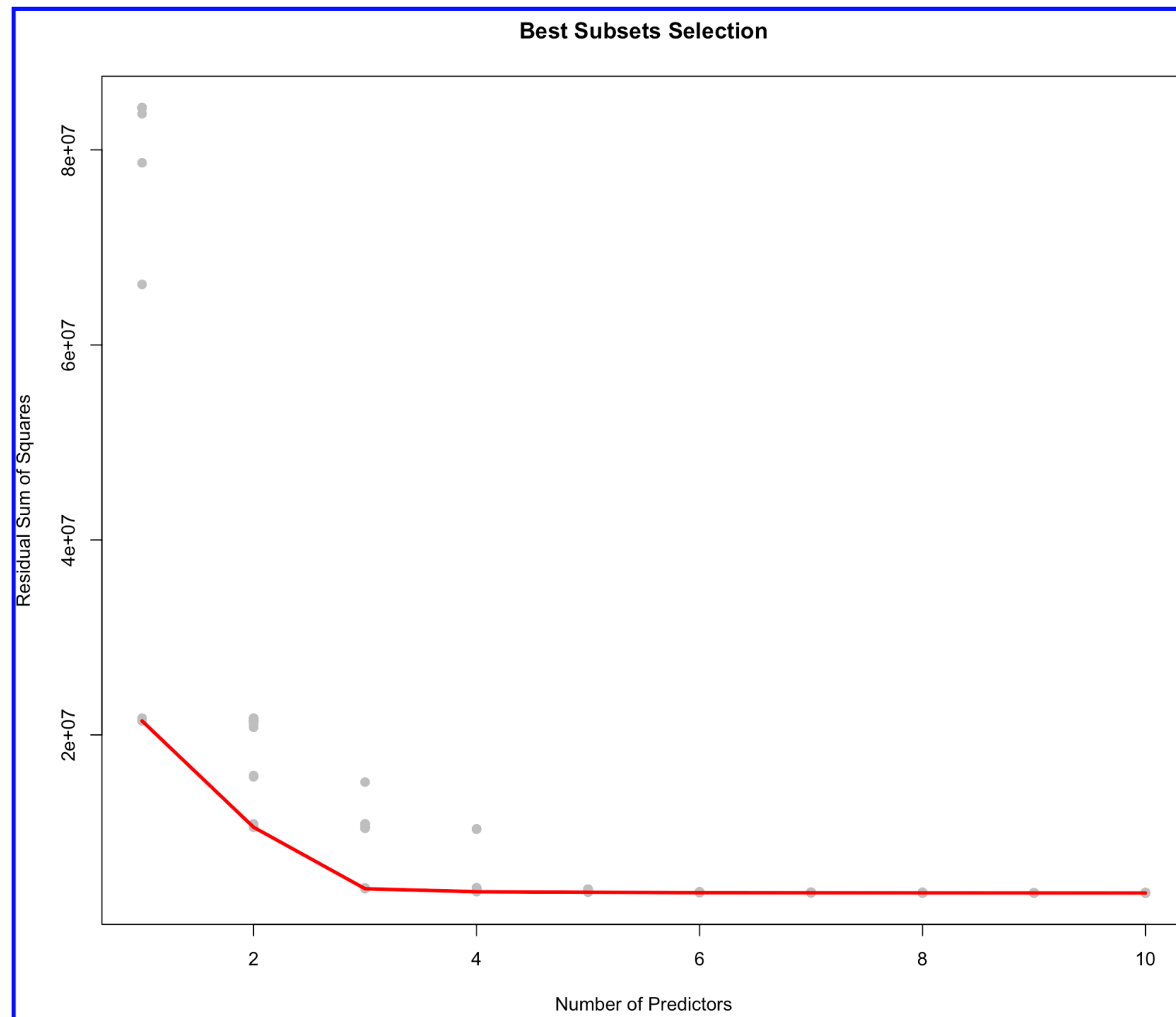
best subset selection, nbest is the number of subsets of each size to record

20 subsets of each size up to 10																
Selection Algorithm: exhaustive																
	ID	Income	Limit	Rating	Cards	Age	Education	GenderFemale	StudentYes	MarriedYes	EthnicityAsian	EthnicityCaucasian				
1 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (2)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (3)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (4)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (5)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (6)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (7)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (8)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (9)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (10)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (11)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
1 (12)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (2)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (3)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (4)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (5)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (6)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (7)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (8)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (9)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (10)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (11)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (12)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (13)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (14)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (15)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (16)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (17)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (18)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (19)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (20)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (2)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (3)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (4)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (5)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (6)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (7)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "

6.1 Subset Selection

Example: credit data

```
#### plot RSS for all models
full.sum<-summary(regfit.full)
plot(as.numeric(row.names(full.sum$which)),full.sum$rss,pch=19,col="grey",xlab="Number of Predictors",ylab="Residual Sum of Squares",main="Best Subsets Selection")
#### plot RSS for best models
regfit.full1<-regsubsets(Balance~.,data=Credit,method="exhaustive",nbest=1,nvmax=10)
full.sum1<-summary(regfit.full1)
lines(1:10,full.sum1$rss,col="red",lwd=3)
```



6.1 Subset Selection

Example: credit data

```
> regfit.full1<-regsubsets(Balance~.,data=Credit,method="exhaustive",nbest=1,nvmax=11)
```

```
> summary(regfit.full1)
```

Subset selection object

Call: regsubsets.formula(Balance ~ ., data = Credit, method = "exhaustive",
nbest = 1, nvmax = 11)

12 Variables (and intercept)

	Forced in	Forced out
ID	FALSE	FALSE
Income	FALSE	FALSE
Limit	FALSE	FALSE
Rating	FALSE	FALSE
Cards	FALSE	FALSE
Age	FALSE	FALSE
Education	FALSE	FALSE
GenderFemale	FALSE	FALSE
StudentYes	FALSE	FALSE
MarriedYes	FALSE	FALSE
EthnicityAsian	FALSE	FALSE
EthnicityCaucasian	FALSE	FALSE

1 subsets of each size up to 11

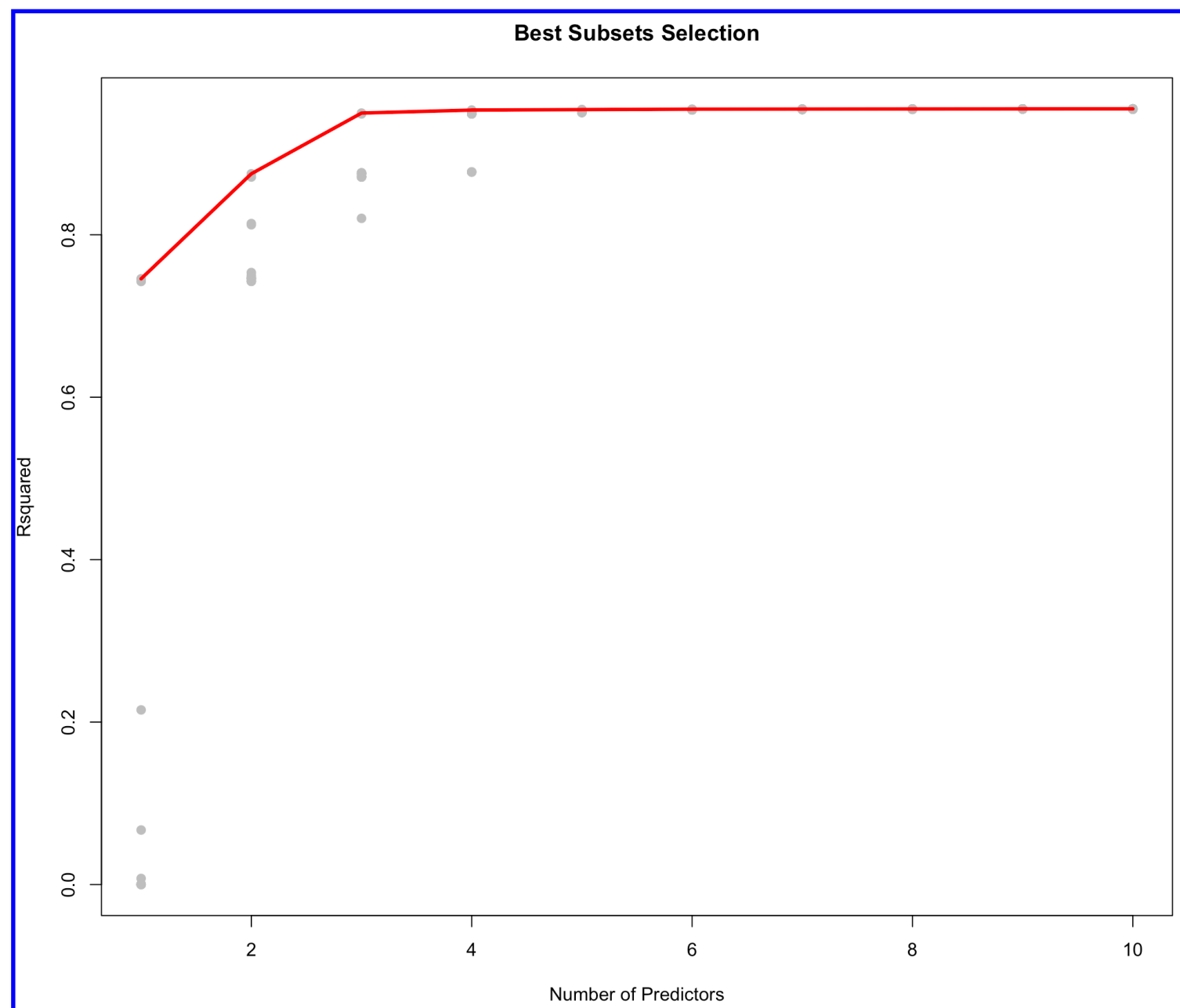
Selection Algorithm: exhaustive

	ID	Income	Limit	Rating	Cards	Age	Education	GenderFemale	StudentYes	MarriedYes	EthnicityAsian	EthnicityCaucasian
1 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
2 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
3 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
4 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
5 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
6 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
7 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
8 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
9 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
10 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "
11 (1)	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "	" "

6.1 Subset Selection

Example: credit data

```
#### plot Rsq for all models
full.sum<-summary(regfit.full)
plot(as.numeric(row.names(full.sum$which)),full.sum$rsq,pch=19,col="grey",xlab="Number of Predictors",ylab="Rsquared",main="Best Subsets Selection")
#### plot Rsq for best models
regfit.full1<-regsubsets(Balance~.,data=Credit,method="exhaustive",nbest=1,nvmax=10)
full.sum1<-summary(regfit.full1)
lines(1:10,full.sum1$rsq,col="red",lwd=3)
```



6.1 Subset Selection

Example: credit data

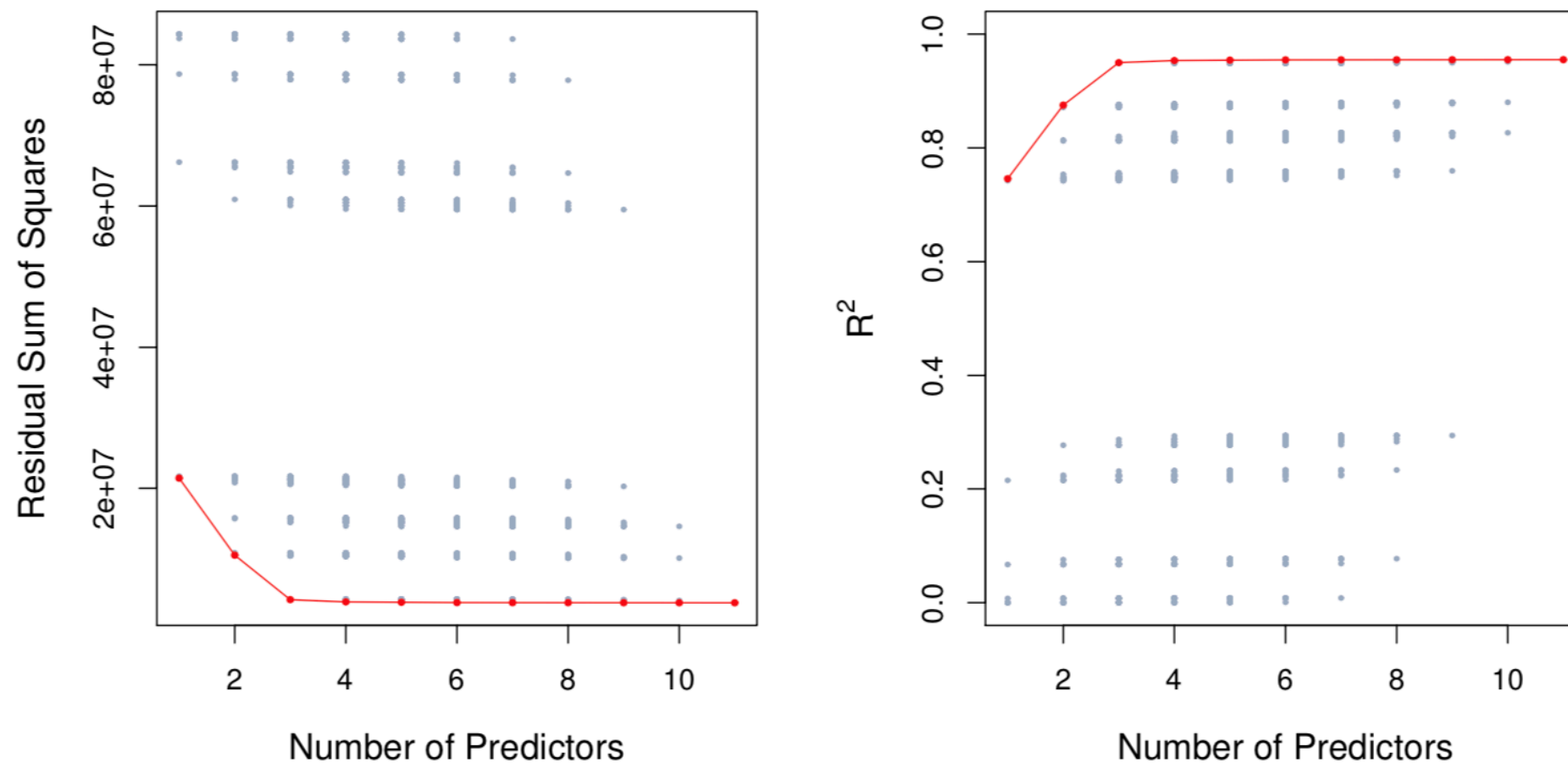


Figure: 6.1. For each possible model containing a subset of the ten predictors in the Credit data set, the RSS and R^2 are displayed. The red frontier tracks the best model for a given number of predictors, according to RSS and R^2 . Though the data set contains only ten predictors, the x-axis ranges from 1 to 11, since one of the variables is categorical and takes on three values, leading to the creation of two dummy variables.

6.1 Subset Selection

The issues of R-squared

▶ *The R-squared is the percentage of the total variation in response due to the inputs.*

▶ *The R-squared reflects the **training error**.*

▶ *However, a model with larger R-squared is not necessarily better than another model with smaller R-squared when we consider **test error**!*

▶ *If model A has all the inputs of model B, then model A's R-squared will always be greater than or as large as that of model B.*

▶ *If model A's additional inputs are entirely uncorrelated with the response, model A contain more noise than model B. As a result, the prediction based on model A would inevitably be poorer or no better.*

6.1 Subset Selection

Adjusted R-squared

The adjusted R-squared, taking into account of the degrees of freedom, is defined as


$$\begin{aligned}\text{adjusted } R^2 &= 1 - \frac{MS_{error}}{MS_{total}} \\ &= 1 - \frac{SS_{error}/(n - p - 1)}{SS_{total}/(n - 1)} \\ &= 1 - \frac{s^2}{\sum_{i=1}^n (y_i - \bar{y})^2 / (n - 1)}\end{aligned}$$

With more inputs, the R^2 always increase, but the adjusted R^2 could decrease since more irrelevant inputs are penalized by the smaller degree of freedom of the residuals.



The adjusted R-squared is preferred over the R-squared in evaluating models.

6.1 Subset Selection

Commonly used criteria for model selection

Akaike information criterion (AIC):

$$\text{AIC} = -2 \log \ell(\hat{\theta}) + 2d$$

where $\hat{\theta}$ is the maximum likelihood estimate, $\ell(\hat{\theta})$ is the likelihood function evaluated at $\hat{\theta}$, and d is number of estimated parameters in the model.

Sometimes, $-2 \log \ell(\hat{\theta})$ is called *Deviance*.

Bayesian information criterion (BIC):

$$\text{BIC} = -2 \log \ell(\hat{\theta}) + \log(n)d$$

6.1 Subset Selection

Criteria given in the reference book

AIC given in the reference book

$$\begin{aligned} \text{AIC} &= \frac{1}{n\hat{\sigma}^2} (\text{RSS} + 2d\hat{\sigma}^2) \\ &= \frac{1}{n} (-2 \log \ell(\hat{\theta}) + 2d). \end{aligned}$$

For linear regression, model parameters are $\hat{\theta} = \{\hat{\beta}, \hat{\sigma}^2\}$, where $\hat{\beta}$ is the fitted regression coefficient.

BIC in the reference book (7th-print)

$$\begin{aligned} \text{BIC} &= \frac{1}{n\hat{\sigma}^2} (\text{RSS} + \log(n)d\hat{\sigma}^2) \\ &= \frac{1}{n} (-2 \log \ell(\hat{\theta}) + \log(n)d) \end{aligned}$$

where $\hat{\sigma}^2$ was missed in first six printed versions.

Mallow's C_p is equivalent to AIC in the special case of Gaussian linear regression.

6.1 Subset Selection

Understanding AIC

Suppose we have models $\mathcal{M}_1, \dots, \mathcal{M}_k$ where each model is a set of densities:

$$\mathcal{M}_j = \{p(y; \theta_j) : \theta_j \in \Theta_j\}.$$

We have data $Y_1, \dots, Y_n$ drawn from some density f . We do not assume that f is in any of the models.

Let $\hat{\theta}_j$ be the MLE from model j . An estimate of p , based on model j is $\hat{p}_j(y) = p(y; \hat{\theta}_j)$. The quality of $\hat{p}_j(y)$ as an estimate of f can be measured by the Kullback-Leibler distance:

$$\begin{aligned} K(p, \hat{p}_j) &= \int p(y) \log \left(\frac{p(y)}{\hat{p}_j(y)} \right) dy \\ &= \int p(y) \log p(y) dy - \int p(y) \log \hat{p}_j(y) dy. \end{aligned}$$

The first term does not depend on j . So minimizing $K(p, \hat{p}_j)$ over j is the same as maximizing

$$K_j = \int p(y) \log \hat{p}_j(y) dy.$$

6.1 Subset Selection

Understanding AIC

We need to estimate K_j . Intuitively, you might think that a good estimate of K_j is

$$\bar{K}_j = \frac{1}{n} \sum_{i=1}^n \log p(Y_i; \hat{\theta}_j) = \frac{\ell_j(\hat{\theta}_j)}{n},$$

where $\ell_j(\theta_j)$ is the log-likelihood function for model j .

However, this estimate is very biased because the data are being used *twice*: first to get the MLE and second to estimate the integral.

Akaike showed that the bias is approximately d_j/n where $d_j = \text{dimension}(\Theta_j)$. Therefore we use

$$\hat{K}_j = \bar{K}_j - \frac{d_j}{n} = \frac{\ell_j(\hat{\theta}_j)}{n} - \frac{d_j}{n}.$$

Therefore, Akaike information Criterion is defined as

$$\text{AIC}_j = -2n\hat{K}_j = -2\ell_j(\hat{\theta}_j) + 2d_j.$$

6.1 Subset Selection

Adjusted R-square, AIC, BIC, Cp in Python

```
# 1. Load the Credit Dataset

Credit = pd.read_csv('Credit.csv')
Credit = Credit.drop('ID', axis=1)
print(Credit.head(10))
[num_samples,num_predictors]=Credit.shape

Credit['Gender'] = Credit['Gender'].map({'Male': 1, 'Female': 0})
Credit['Student'] = Credit['Student'].map({'Yes': 1, 'No': 0})
Credit['Married'] = Credit['Married'].map({'Yes': 1, 'No': 0})
Credit['Ethnicity'] = Credit['Ethnicity'].map({'Asian': 2, 'Caucasian': 1, 'African American': 0})

# 2. Scatterplot Matrix
selected_columns=['Balance', 'Education', 'Age', 'Cards', 'Rating', 'Limit', 'Income']
# Plot scatterplot matrix
sns.set(style="ticks", color_codes=True)
plt.figure(figsize=(12, 10))
sns.pairplot(Credit[selected_columns], diag_kind='kde')
plt.suptitle("Scatterplot Matrix", y=1.02)
plt.show()

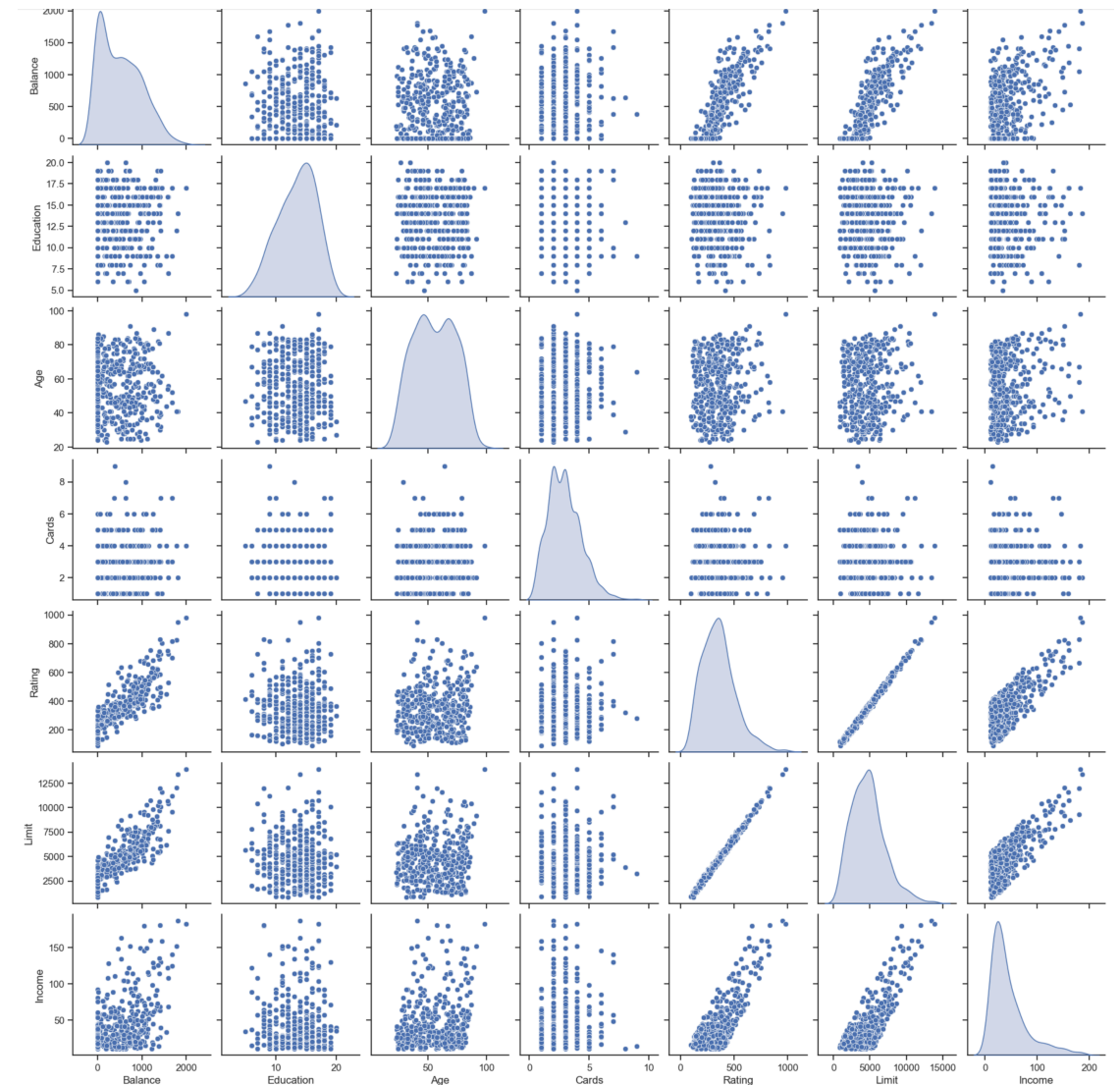
# 3. Best Subset Selection
# -----
# We'll perform exhaustive best subset selection for models with up to 10 predictors.

# Define target variable and predictors
target = 'Balance'
predictors = [col for col in Credit.columns if col != target]
print(predictors)
# Set maximum number of predictors
nvmax = 10

# Initialize lists to store model statistics
model_stats = []

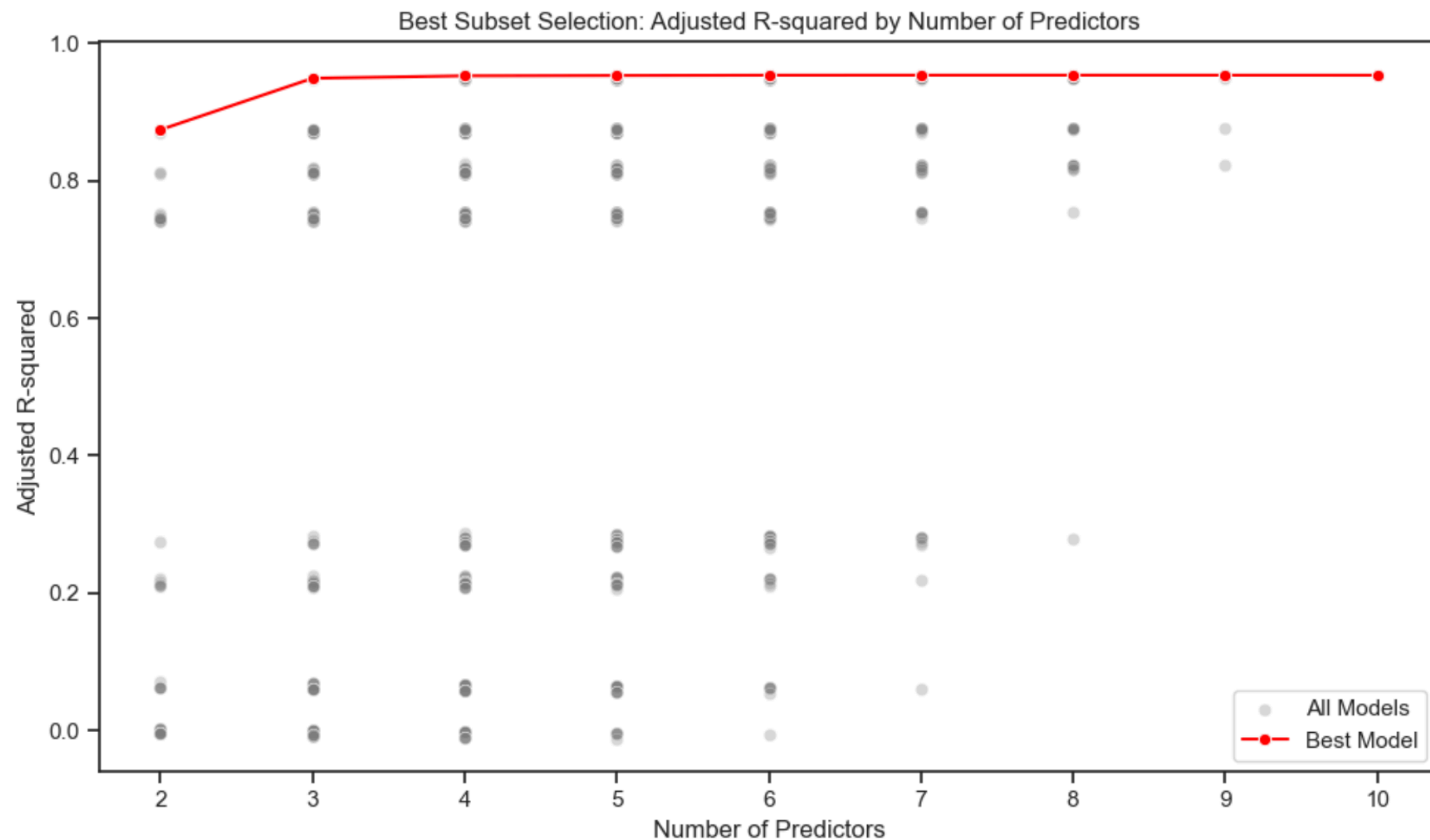
# Iterate over number of predictors from 1 to nvmax
for k in range(nvmax, 1, -1):
    print(f"Evaluating models with {k} predictors...")
    # Generate all possible combinations of predictors of size k
    subsets = list(combinations(predictors, k))
    for subset in subsets:
        # Prepare the design matrix
        X_subset = Credit[list(subset)]
        X_subset = sm.add_constant(X_subset) # Adds a constant term to the predictor
        # Fit the model using Ordinary Least Squares
        model = sm.OLS(Credit[target], X_subset).fit()

        # Calculate RSS
        rss = sum(model.resid ** 2)
```



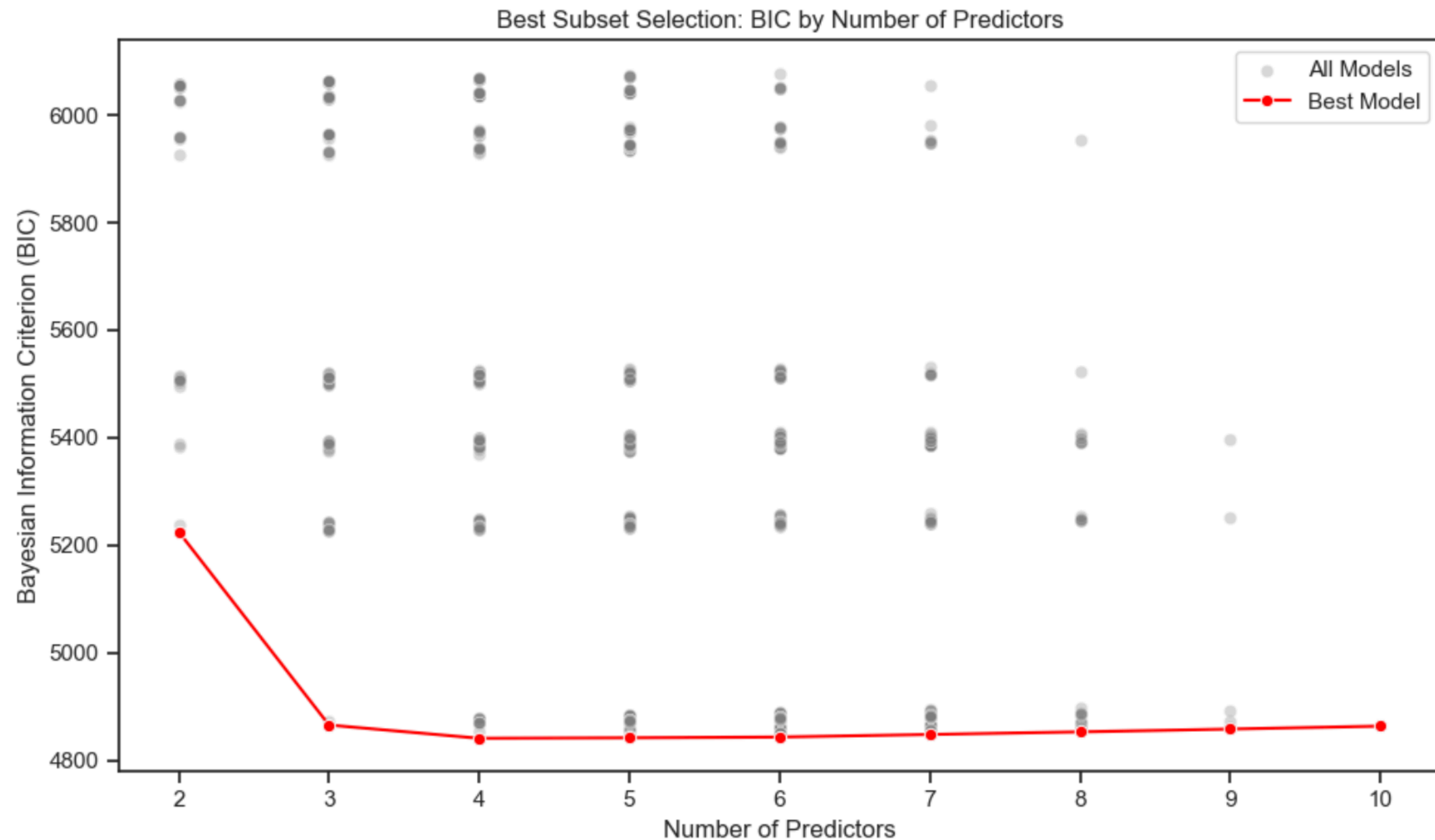
6.1 Subset Selection

Adjusted R-square, AIC, BIC, Cp in Python



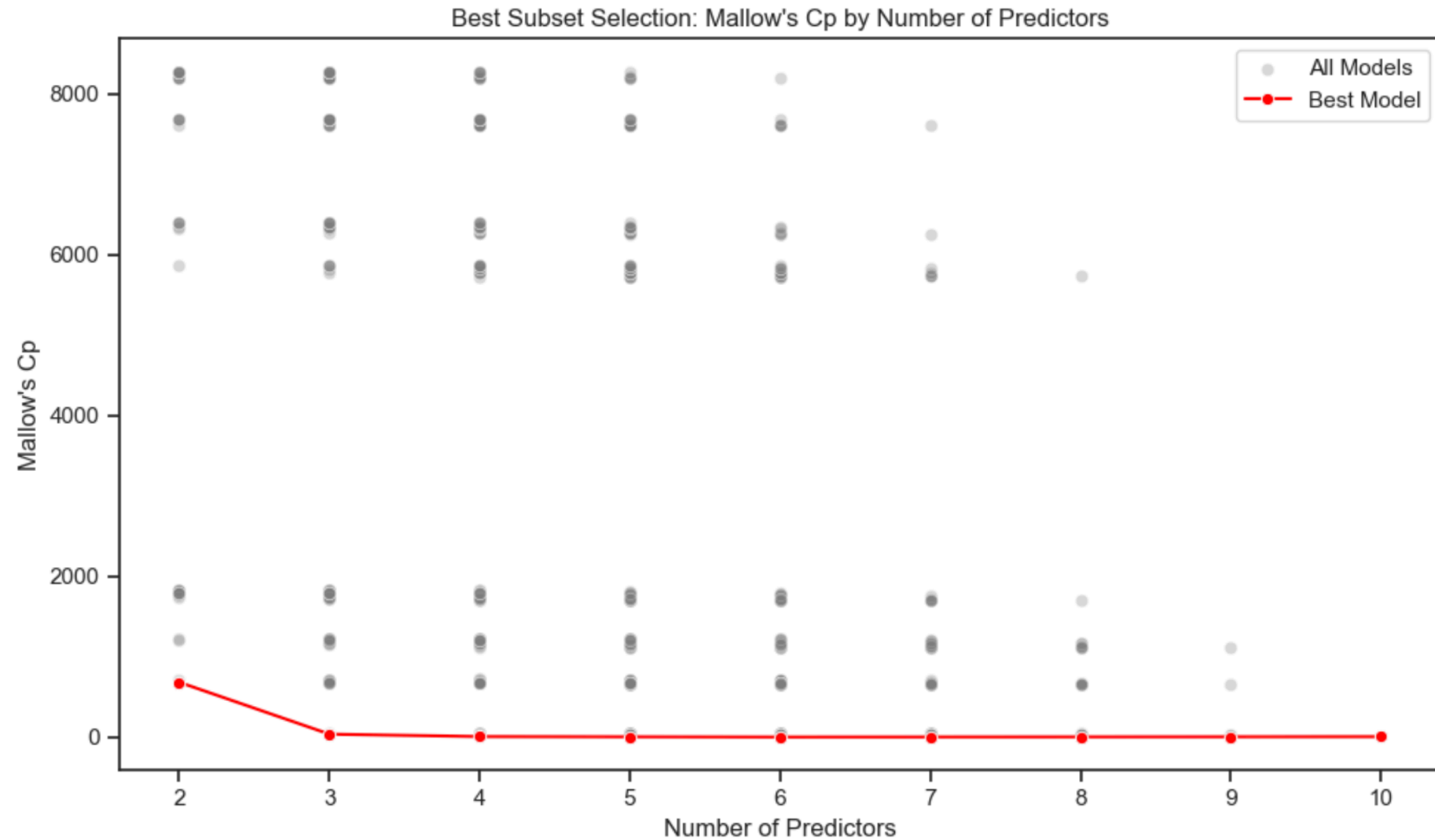
6.1 Subset Selection

Adjusted R-square, AIC, BIC, Cp in Python



6.1 Subset Selection

Adjusted R-square, AIC, BIC, Cp in Python



6.1 Subset Selection

Adjusted R-square, AIC, BIC, Cp in Python

Best Models for Each Number of Predictors:

	Num_Predictors	Predictors \
969	2	(Income, Rating)
860	3	(Income, Rating, Student)
648	4	(Income, Limit, Cards, Student)
389	5	(Income, Limit, Rating, Cards, Student)
178	6	(Income, Limit, Rating, Cards, Age, Student)
64	7	(Income, Limit, Rating, Cards, Age, Student, E...
18	8	(Income, Limit, Rating, Cards, Age, Gender, St...
5	9	(Income, Limit, Rating, Cards, Age, Gender, St...
0	10	(Income, Limit, Rating, Cards, Age, Education,...

	RSS	R_squared	Adj_R_squared	BIC	Cp
969	1.053254e+07	0.875118	0.874489	5224.531479	685.895393
860	4.227219e+06	0.949879	0.949499	4865.352851	40.217059
648	3.915058e+06	0.953580	0.953110	4840.658660	10.152112
389	3.866091e+06	0.954161	0.953579	4841.615607	7.122228
178	3.821620e+06	0.954688	0.953996	4842.979215	4.554143
64	3.810489e+06	0.954820	0.954013	4847.803930	5.410780
18	3.799083e+06	0.954955	0.954033	4852.596263	6.239154
5	3.791716e+06	0.955042	0.954005	4857.811391	7.482497
0	3.787019e+06	0.955098	0.953944	4863.307022	9.000000

6.1 Subset Selection

Pros and Cons of best subset selection

- ▶ *Seems straightforward to carry out.*
- ▶ *Conceptually clear.*
- ▶ *The search space too large (2^p models), may lead to overfit.*
- ▶ *Computationally infeasible: too many models to run.*
- ▶ *if $p = 20$, there are $2^{20} > 1,000,000$ models.*

6.1 Subset Selection

Forward stepwise selection

- ▶ *Start with the null model.*
- ▶ *Find the best one-variable model.*
- ▶ *With the best one-variable model, add one more variable to get the best two-variable model.*
- ▶ *With the best two-variable model, add one more variable to get the best three-variable model.*
- • •
- ▶ *Find the best among all these best k -variable models.*

6.1 Subset Selection

Algorithm of forward stepwise selection

Step 1. Let $\mathcal{M}_0$ be the null model, $Y = \beta_0 + \epsilon$. The predictor is the sample mean of response.

Step 2. For $k = 0, 1, \dots, p - 1$,
Consider all $p - k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
Choose the best among these $p - k$ models, and call it $\mathcal{M}_{k+1}$. Here best is defined as having smallest RSS or highest R^2 .

Step 3. Select a single best model from $\mathcal{M}_0, \dots, \mathcal{M}_p$ by cross validation or AIC or BIC or C_p or adjusted R^2 .

6.1 Subset Selection

Pros and cons of forward stepwise selection

- ▶ *Less computation*
- ▶ *Less models ($\sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ models).*
- ▶ *If $p = 20$, only 211 models, compared with more than 1 million models for best subset selection*
- ▶ *No problem for first n -steps if $p > n$.*
- ▶ *Once an input is in, it does not get out.*

6.1 Subset Selection

Example: credit dataset

Variables	Best subset	Forward stepwise
one	rating	rating
two	rating, income	rating, income
three	rating, income, student	rating, income, student
four	cards, income, student, limit	rating, income, student, limit

TABLE 6.1. The first four selected models for best subset selection and forward stepwise selection on the Credit data set. The first three models are identical but the fourth models differ.

6.1 Subset Selection

Forward Subset Selection

Example: credit dataset

```
##### Forward Subset Selection
```

```
regfit.fwd<-regsubsets(Balance~.,data=Credit,method="forward",nvmax=11)
summary(regfit.fwd)
```

```
## forward subset selection
```

```
1 subsets of each size up to 11
```

```
Selection Algorithm: forward
```

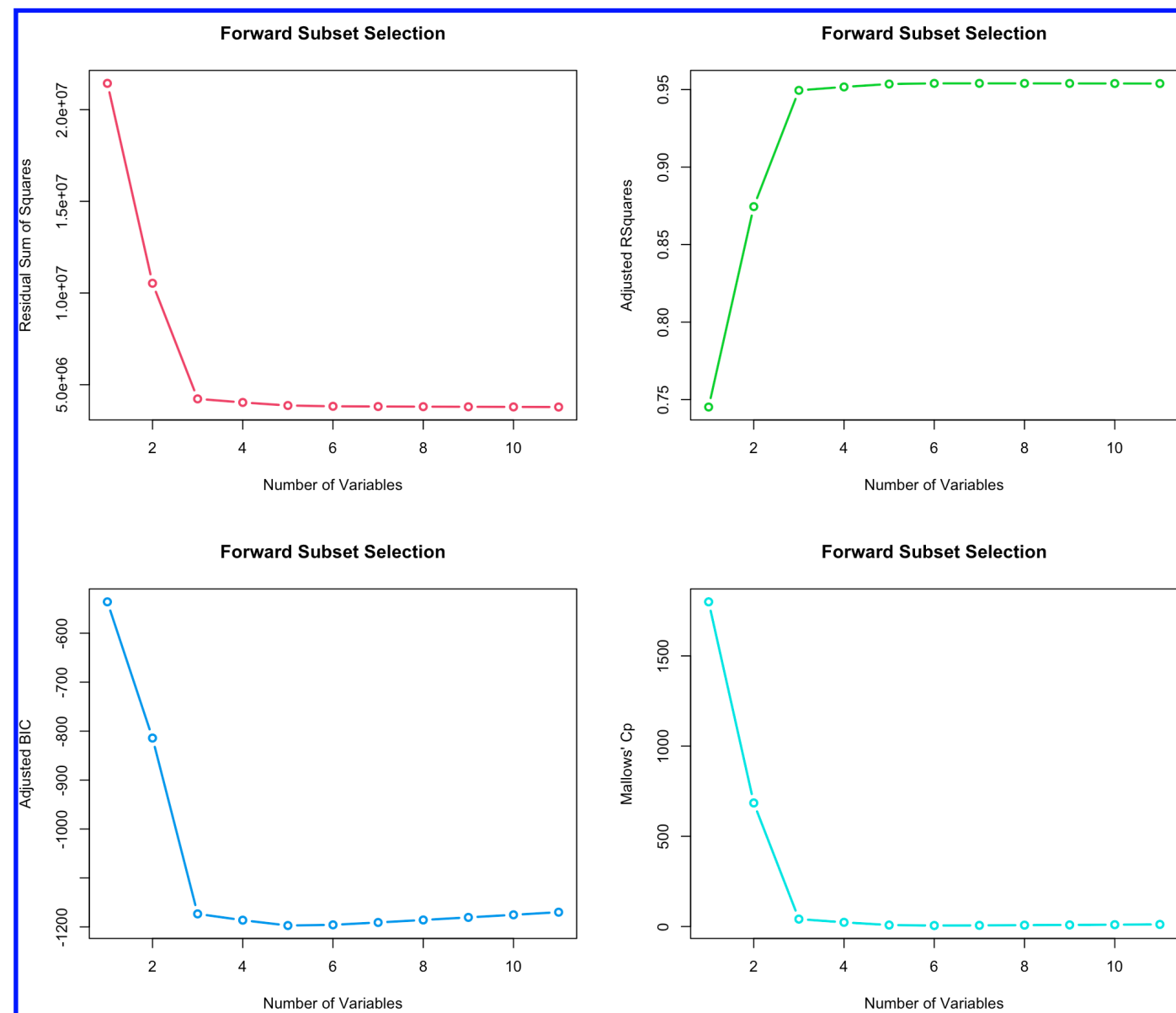
		ID	Income	Limit	Rating	Cards	Age	Education	GenderFemale	StudentYes	MarriedYes	EthnicityAsian	EthnicityCaucasian
1	(1)	" "	" "	" "	"*"	" "	" "	" "	" "	" "	" "	" "	" "
2	(1)	" "	"*"	" "	"*"	" "	" "	" "	" "	" "	" "	" "	" "
3	(1)	" "	"*"	" "	"*"	" "	" "	" "	" "	"*"	" "	" "	" "
4	(1)	" "	"*"	"*"	"*"	" "	" "	" "	" "	"*"	" "	" "	" "
5	(1)	" "	"*"	"*"	"*"	"*"	" "	" "	" "	"*"	" "	" "	" "
6	(1)	" "	"*"	"*"	"*"	"*"	"*"	" "	" "	"*"	" "	" "	" "
7	(1)	" "	"*"	"*"	"*"	"*"	"*"	" "	"*"	"*"	" "	" "	" "
8	(1)	"*"	"*"	"*"	"*"	"*"	"*"	" "	"*"	"*"	" "	" "	" "
9	(1)	"*"	"*"	"*"	"*"	"*"	"*"	" "	"*"	"*"	" "	"*"	" "
10	(1)	"*"	"*"	"*"	"*"	"*"	"*"	" "	"*"	"*"	"*"	"*"	" "
11	(1)	"*"	"*"	"*"	"*"	"*"	"*"	" "	"*"	"*"	"*"	"*"	"*"

6.1 Subset Selection

Forward Subset Selection

Example: credit dataset

```
fwd.sum<-summary(regfit.fwd)
par(mfrow=c(2,2))
plot(1:11,fwd.sum$rss,type="b",col=2,lwd=2,xlab="Number of Variables",ylab="Residual Sum of Squares",main="Forward Subset Selection")
plot(1:11,fwd.sum$adjr2,type="b",col=3,lwd=2,xlab="Number of Variables",ylab="Adjusted RSquares",main="Forward Subset Selection")
plot(1:11,fwd.sum$bic,type="b",col=4,lwd=2,xlab="Number of Variables",ylab="Adjusted BIC",main="Forward Subset Selection")
plot(1:11,fwd.sum$cp,type="b",col=5,lwd=2,xlab="Number of Variables",ylab="Mallows' Cp",main="Forward Subset Selection")
```



6.1 Subset Selection

Backward stepwise selection

- ▶ *Start with the largest model (all p inputs in).*
- ▶ *Find the best $(p-1)$ -variable model, by reducing one from the largest model*
- ▶ *Find the best $(p-2)$ -variable model, by reducing one variable from the best $(p-1)$ -variable model.*
- ▶ *Find the best $(p-3)$ -variable model, by reducing one variable from the best $(p-2)$ -variable model.*
- ● ●
- ▶ *Find the best 1-variable model, by reducing one variable from the best 2-variable model.*
- ▶ *The null model .*

6.1 Subset Selection

Algorithm of backward stepwise selection

Step 1. Let $\mathcal{M}_p$ be the full model.

Step 2. For $k = p, p - 1, \dots, 1$,
Consider all k models that contain all but one of the
predictors in $\mathcal{M}_k$ for a total of $k - 1$ predictors
Choose the best among these k models, and call it $\mathcal{M}_{k-1}$.
Here best is defined as having smallest RSS or highest R^2 .

Step 3. Select a single best model from $\mathcal{M}_0, \dots, \mathcal{M}_p$ by
cross validation or AIC or BIC or C_p or adjusted R^2 .

6.1 Subset Selection

Pros and cons of backward stepwise selection

- ▶ *Less computation*
- ▶ *Less models ($\sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ models).*
- ▶ *If $p = 20$, only 211 models, compared with more than 1 million models for best subset selection*
- ▶ *No applicable to the case with $p > n$.*
- ▶ *Once an input is out, it does not get in.*

6.1 Subset Selection

Example by Credit Data

Backward Subset Selection

```
##### Backward Subset Selection
```

```
regfit.bwd<-regsubsets(Balance~.,data=Credit,method="backward",nvmax=11)  
summary(regfit.bwd)
```

```
## backward subset selection
```

1 subsets of each size up to 11

Selection Algorithm: backward

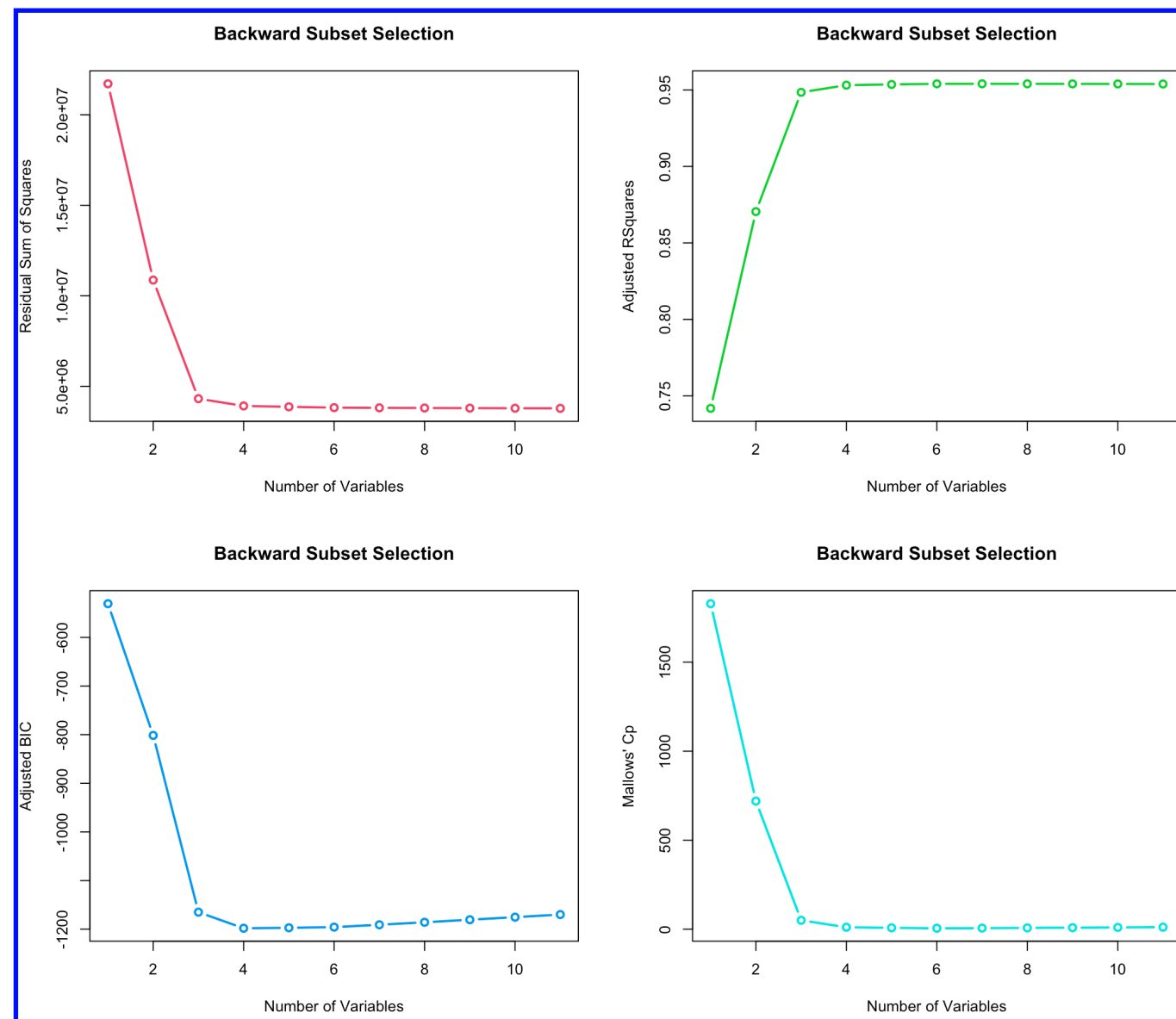
		ID	Income	Limit	Rating	Cards	Age	Education	GenderFemale	StudentYes	MarriedYes	EthnicityAsian	EthnicityCaucasian
1	(1)	" "	" "	"*	" "	" "	" "	" "	" "	" "	" "	" "	" "
2	(1)	" "	"*	"*	" "	" "	" "	" "	" "	" "	" "	" "	" "
3	(1)	" "	"*	"*	" "	" "	" "	" "	"*	" "	" "	" "	" "
4	(1)	" "	"*	"*	" "	"*	" "	" "	" "	"*	" "	" "	" "
5	(1)	" "	"*	"*	"*	"*	" "	" "	" "	"*	" "	" "	" "
6	(1)	" "	"*	"*	"*	"*	"*	" "	" "	"*	" "	" "	" "
7	(1)	" "	"*	"*	"*	"*	"*	" "	"*	"*	" "	" "	" "
8	(1)	"*	"*	"*	"*	"*	"*	" "	"*	"*	" "	" "	" "
9	(1)	"*	"*	"*	"*	"*	"*	" "	"*	"*	" "	"*	" "
10	(1)	"*	"*	"*	"*	"*	"*	" "	"*	"*	"*	"*	" "
11	(1)	"*	"*	"*	"*	"*	"*	" "	"*	"*	"*	"*	"*

6.1 Subset Selection

Example by Credit Data

Backward Subset Selection

```
bwd.sum<-summary(regfit.bwd)
par(mfrow=c(2,2))
plot(1:11,bwd.sum$rss,type="b",col=2,lwd=2,xlab="Number of Variables",ylab="Residual Sum of Squares",main="Backward Subset Selection")
plot(1:11,bwd.sum$adjr2,type="b",col=3,lwd=2,xlab="Number of Variables",ylab="Adjusted RSquares",main="Backward Subset Selection")
plot(1:11,bwd.sum$bic,type="b",col=4,lwd=2,xlab="Number of Variables",ylab="Adjusted BIC",main="Backward Subset Selection")
plot(1:11,bwd.sum$cp,type="b",col=5,lwd=2,xlab="Number of Variables",ylab="Mallows' Cp",main="Backward Subset Selection")
```



6.1 Subset Selection

Find the best model based on prediction error

Validation/cross-validation approach (addressed in Chapter 5).

Use Adjusted R^2 , AIC, BIC or C_p .

Example

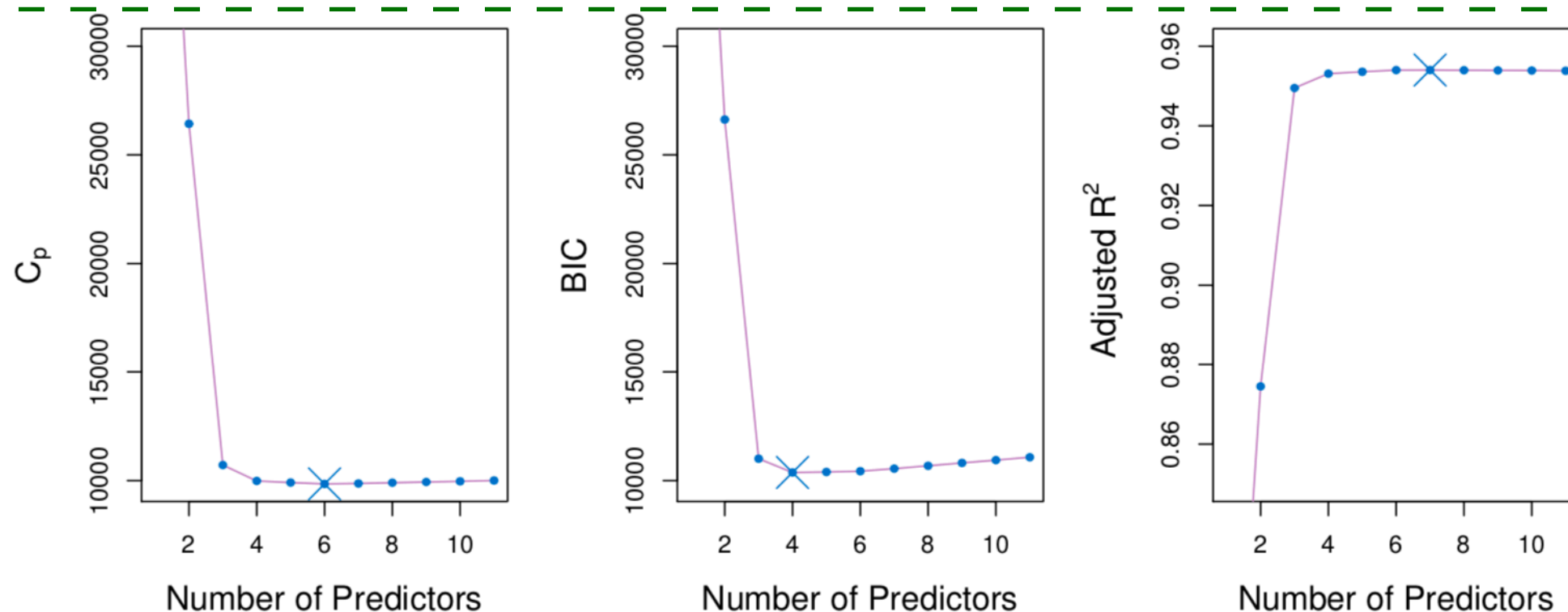


Figure: 6.2. C_p , BIC, and adjusted R^2 are shown for the best models of each size for the Credit data set (the lower frontier in Figure 6.1). C_p and BIC are estimates of test MSE. In the middle plot we see that the BIC estimate of test error shows an increase after four variables are selected. The other two plots are rather flat after four variables are included.

6.1 Subset Selection

Example

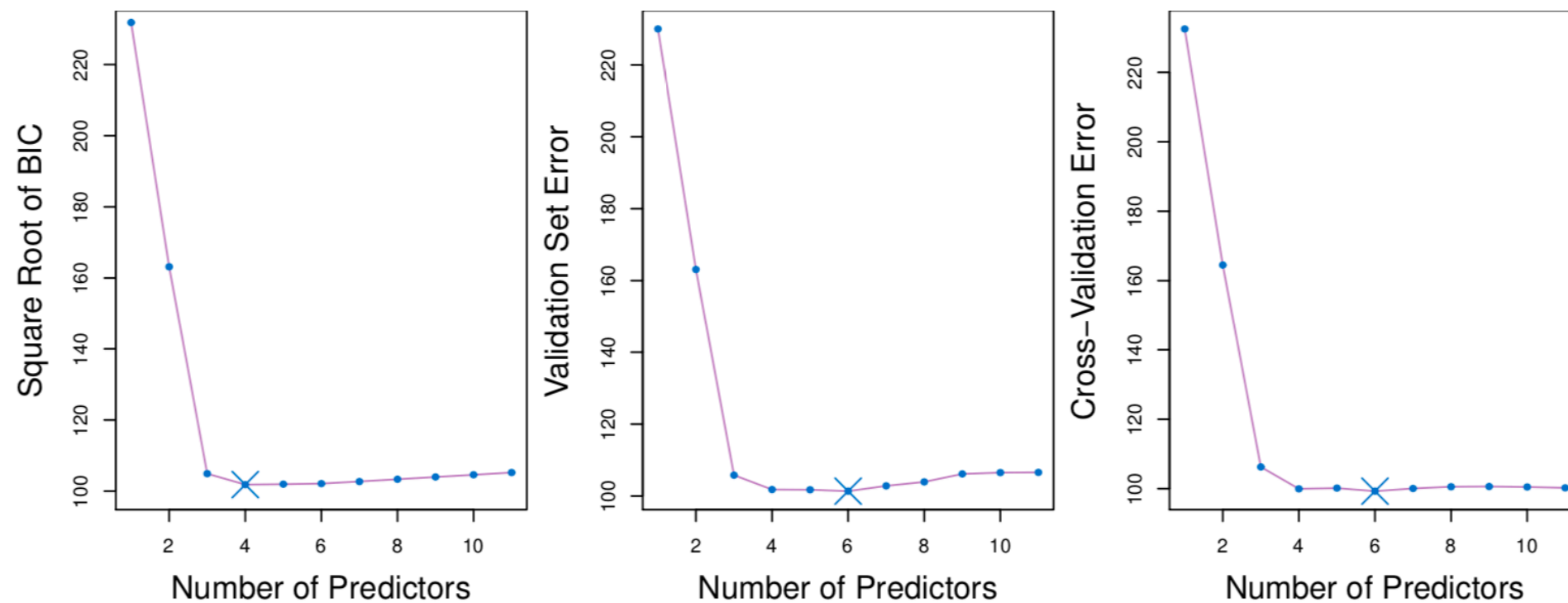
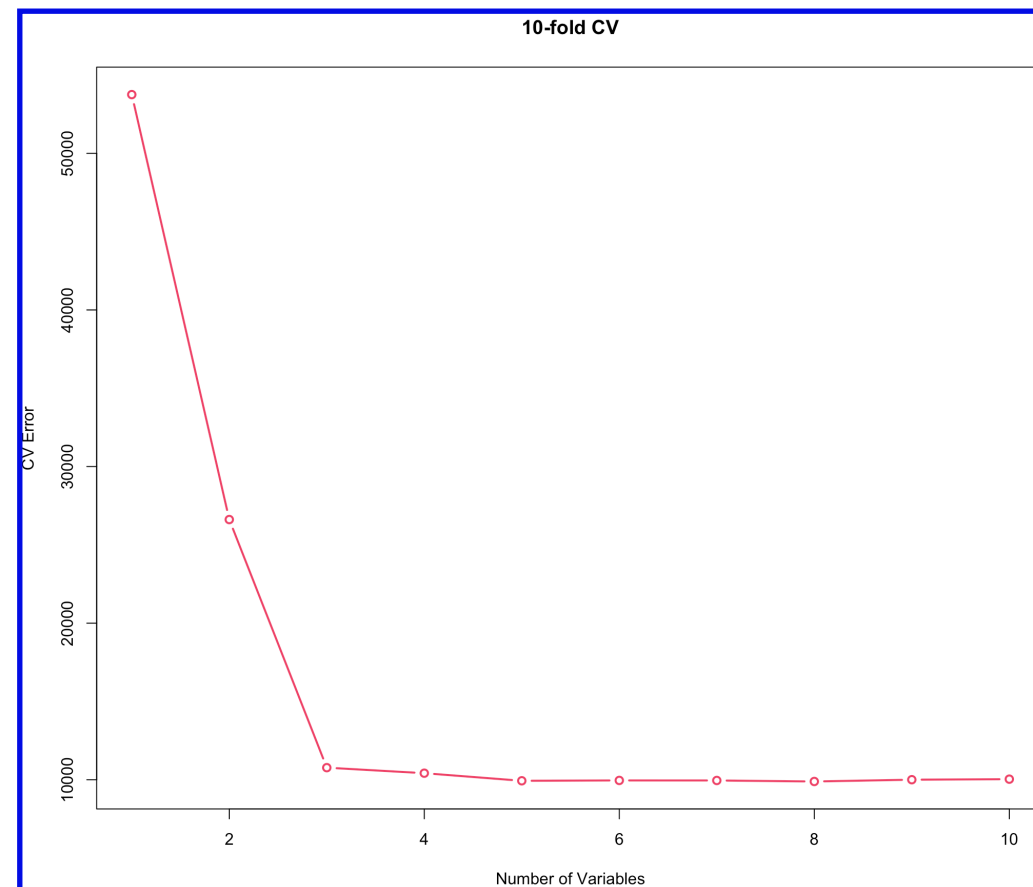


Figure: 6.3. For the Credit data set, three quantities are displayed for the best model containing d predictors, for d ranging from 1 to 11. The overall best model, based on each of these quantities, is shown as a blue cross. Left: Square root of BIC. Center: Validation set errors (75% training data). Right: 10-fold Cross-validation errors.

6.1 Subset Selection

10-fold CV Errors

```
> ##### 10-fold CV Errors of Best Models by Forward Subset Selections
> library(boot)
> CV10.err<-rep(0,10)
> for(p in 1:10){
+   x<-which(summary(regfit.fwd)$which[p,])
+   x<-as.numeric(x)
+   x<-x[-1]-1
+   dfname<-c("Balance",names(Credit)[x])
+   newCred<-Credit[,dfname]
+
+   glm.fit=glm(Balance~. ,data=newCred)
+   cv.err=cv.glm(newCred,glm.fit,K=10)
+   CV10.err[p]=cv.err$delta[1]
+ }
> CV10.err
[1] 53750.427 26614.488 10772.685 10419.816 9934.294 9957.340 9954.611 9889.670 10003.669 10038.354
> plot(1:10,CV10.err,type="b",lwd=2,col=2,xlab="Number of Variables",ylab="CV Error",main="10-fold CV")
```



6.1 Subset Selection

The one standard deviation rule

▶ *In the above figure, model with 6 inputs do not seem to be much better than model with 4 or 3 inputs.*

▶ *Keep in mind the Occam's razor: Choose the simplest model if they are similar by other criterion.*

▶ *Calculate the standard error of the estimated test MSE for each model size*

▶ *Consider the models with estimated test MSE of one standard deviation within the smallest test MSE.*

▶ *Among them select the one with the smallest model size.*

▶ *Apply this rule to the Example in Figure 6.3 gives the model with 3 variable.*

6.2 Shrinkage Methods by Regularization

Ridge regression

▶ The least squares estimator $\hat{\beta}$ is minimizing

$$\text{RSS} = \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2$$

▶ The ridge regression $\hat{\beta}_\lambda^R$ is minimizing

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p \beta_j^2$$

where $\lambda \geq 0$ is a tuning parameter.

▶ *The first term measures goodness of fit, the smaller the better.*

▶ The second term $\lambda \sum_{j=1}^p \beta_j^2$ is called shrinkage penalty, which *shrinks* β_j towards 0.

▶ *The shrinkage reduces variance (at the cost of increased bias) !*

6.2 Shrinkage Methods by Regularization

Ridge regression

DISCUSSION

What is the solution of Ridge regression?

What is the bias and the variance?

6.2 Shrinkage Methods by Regularization

Tuning parameter λ

- ▶ $\lambda = 0$: no penalty, $\hat{\beta}_0^R = \hat{\beta}$.
- ▶ $\lambda = \infty$: infinity penalty, $\hat{\beta}_\lambda^R = 0$.
- ▶ Large λ : heavy penalty, more shrinkage of the estimator.
- ▶ Note that β_0 is not penalized.

6.2 Shrinkage Methods by Regularization

Standardize the inputs

For j -th input X_j with observations: $(x_{1j}, \dots, x_{nj})$, standardize it as

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\sqrt{(1/n) \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}}$$

to get rid of the scale of X_j .

Suggest to apply standardization before trying ridge regression.

Least squares is unaffected by the scale of X_j . (i.e.,
 $X_j \hat{\beta}_j = (cX_j)(\hat{\beta}_j/c)$)

Ridge is affected by λ as well as the scale of the inputs.

6.2 Shrinkage Methods by Regularization

Example on Credit Data

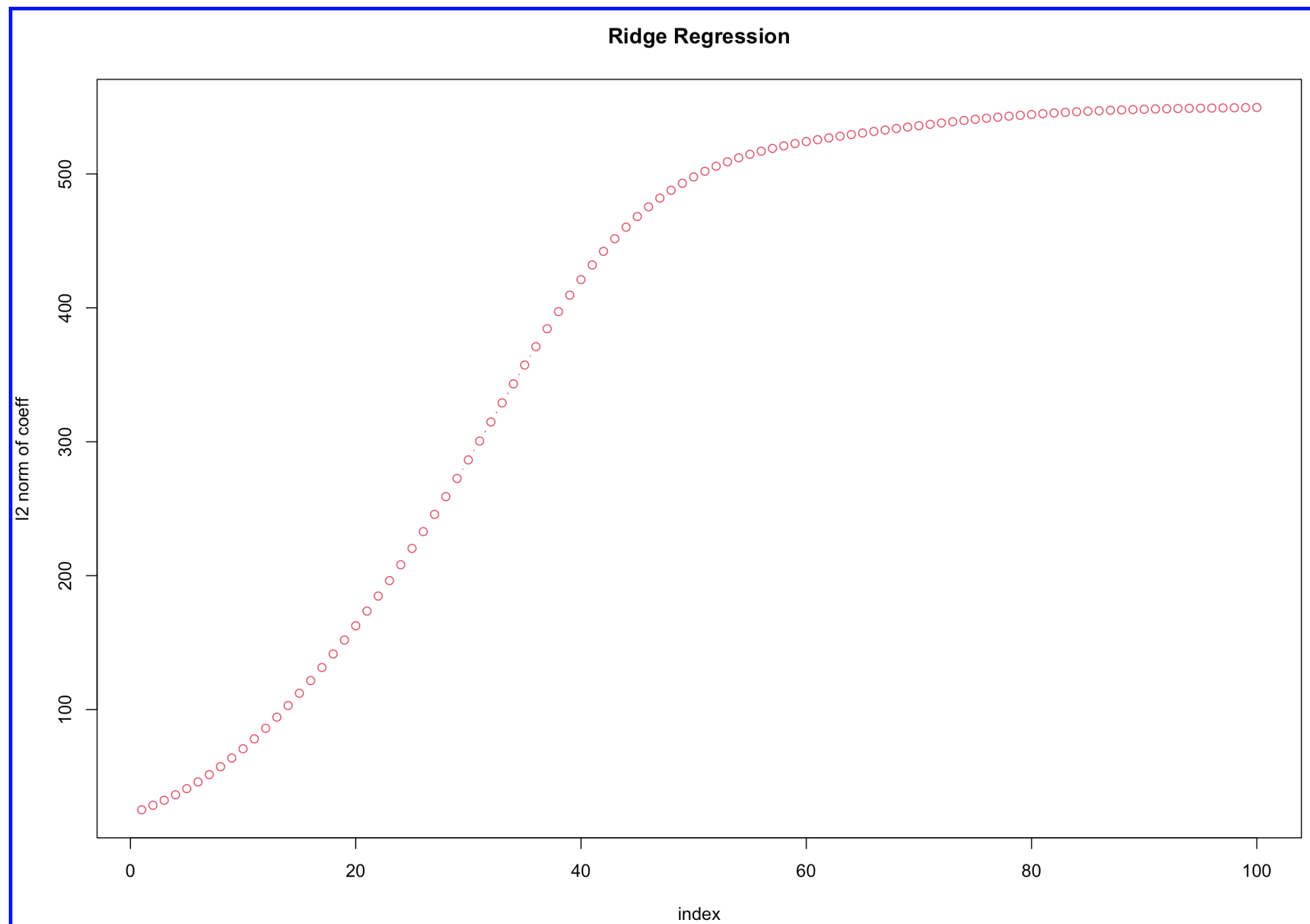
```
> ##### Ridge Regression
> x=scale(model.matrix(Balance~.,Credit)[-1]) ## set up the covariate matrix and standardize the variables
> y=Credit$Balance ## set up the response vector
>
> ##We will use the glmnet package in order to perform ridge regression and the lasso.
> library(glmnet)
> grid=10^seq(4,-2,length=100) ## set up the possible values for lambda (regularization parameter)
> ridge.mod=glmnet(x,y,alpha=0,lambda=grid) ## alpha=0 for ridge regression; alpha=1 for LASSO
> names(ridge.mod) ## results contained in the summary
[1] "a0" "beta" "df" "dim" "lambda" "dev.ratio" "nulldev" "npasses" "jerr" "offset" "call" "nobs"
>
> ridge.mod$lambda[50] ## the 50-th lambda value
[1] 10.72267
> coef(ridge.mod)[,50] ## coefficient for the 50-th model
      (Intercept)          ID          Income          Limit          Rating          Cards          Age          Education          GenderFemale          StudentYes
      520.015000      4.405020     -244.536470      306.126875      279.721729      20.457489     -13.106079      -2.381980      -3.868313      123.533026
      MarriedYes EthnicityAsian EthnicityCaucasian
      -5.458761      7.227021      4.622668
> sqrt(sum(coef(ridge.mod)[-1,50]^2)) ## l2 norm of the above coefficients
[1] 497.745
>
> ridge.mod$lambda[60] ## the 60-th lambda value
[1] 2.656088
> coef(ridge.mod)[,60] ## coefficient for the 60-th model
      (Intercept)          ID          Income          Limit          Rating          Cards          Age          Education          GenderFemale          StudentYes
      520.015000      4.717768     -267.093304      332.071794      276.486050      20.296840     -11.544035      -2.783771      -4.537786      126.403648
      MarriedYes EthnicityAsian EthnicityCaucasian
      -5.128536      7.685553      4.687790
> sqrt(sum(coef(ridge.mod)[-1,60]^2)) ## l2 norm of the above coefficients
[1] 524.1527
```

Ridge regression with different values of λ

6.2 Shrinkage Methods by Regularization

Example on Credit Data

```
plot(sqrt(colSums(coef(ridge.mod)[-1,]^2)),xlab="index",ylab="l2 norm of coeff",type='b',col=2,main="Ridge Regression")
```

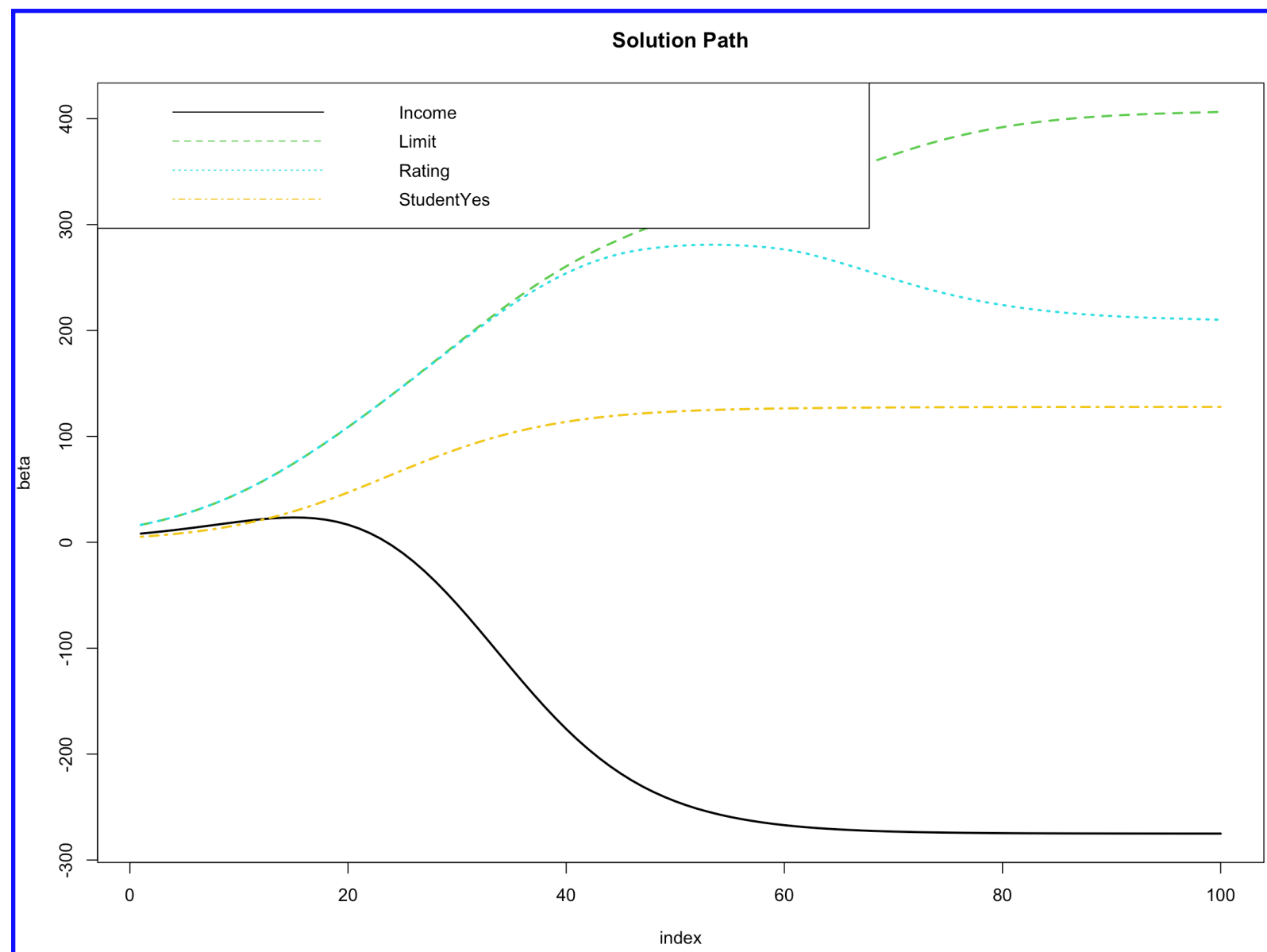


6.2 Shrinkage Methods by Regularization

Example on Credit Data

```
## plot solution path
matplot(t(coef(ridge.mod)[c(3,4,5,10),]), type = "l", col = c(1,3,5,7), lty = c(1,2,3,4), lwd = 2, xlab="index", main="Solution Path", ylab="beta")
legend("topleft",
      legend = c("Income", "Limit", "Rating", "StudentYes"),
      col = c(1,3,5,7),
      lty=c(1,2,3,4)
)
```

Estimated coefficients with different values of λ



6.2 Shrinkage Methods by Regularization

Example

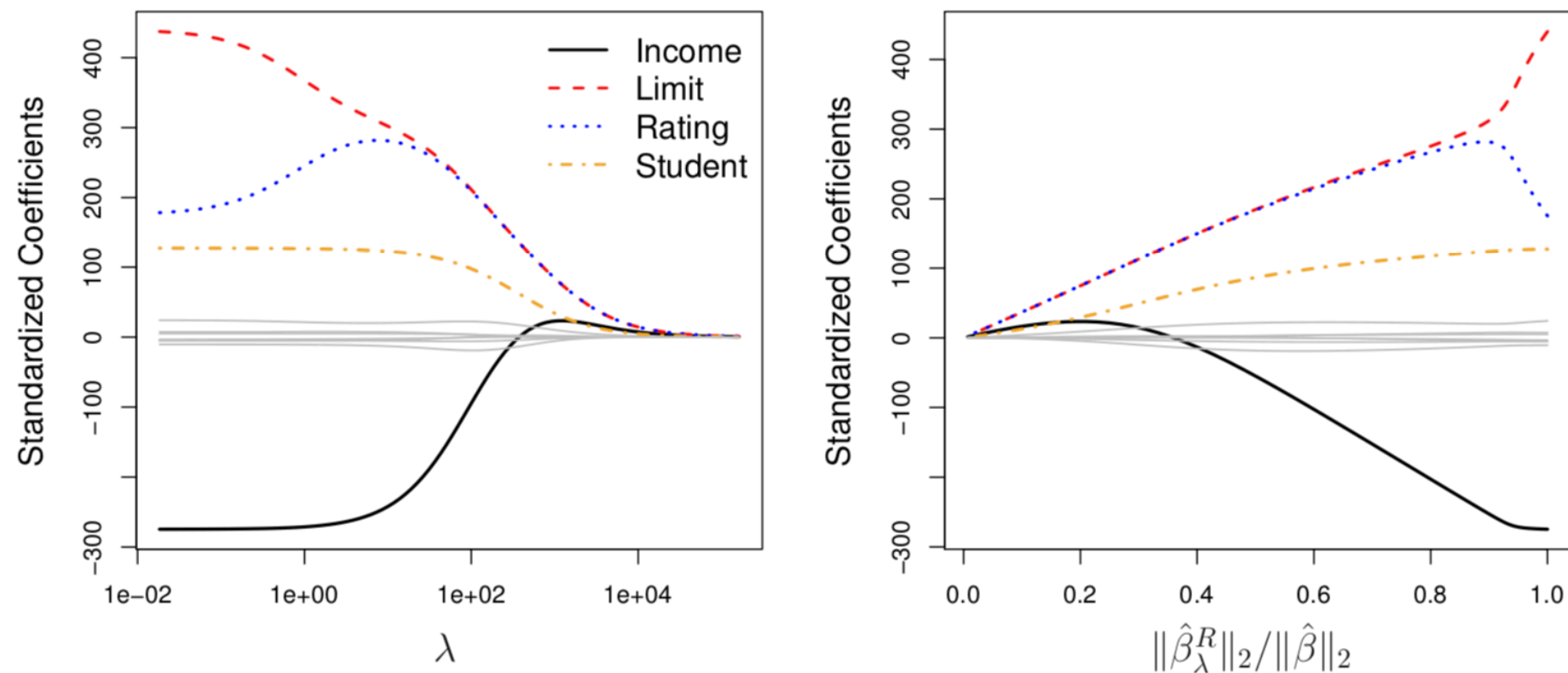


Figure: 6.4. The standardized ridge regression coefficients are displayed for the Credit data set, as a function of λ and $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$.

Here $\|\mathbf{a}\|_2 = \sqrt{\sum_{j=1}^p a_j^2}$.

6.2 Shrinkage Methods by Regularization

Example

```
> ## We can use the predict() function for a number of purposes.
> predict(ridge.mod,s=50,type="coefficients")[1:13,]      ##obtain the ridge regression coefficients for a new value of lambda, say 50:
(Intercept)      ID      Income      Limit      Rating      Cards      Age      Education      GenderFemale      StudentYes
520.015000      3.218591      -165.567437      254.251576      248.721476      22.104604      -17.442588      -1.259881      -1.570771      111.990014
MarriedYes      EthnicityAsian      EthnicityCaucasian
-6.060871      5.345378      4.170971
>
> set.seed(1)
> train=sample(1:nrow(x), nrow(x)/2)
> test=(-train)
> y.test=y[test]      ## Split into train and test
>
> ridge.mod=glmnet(x[train,],y[train],alpha=0,lambda=grid, thresh =1e-12)  ## ridge regression with different lambda values on train dataset
> ridge.pred=predict(ridge.mod,s=4,newx=x[test,])      ## predict on test data with lambda=4
> mean((ridge.pred-y.test)^2)      ## average test error
[1] 10293.33
>
>
> ridge.pred=predict(ridge.mod,s=1e10,newx=x[test,])      ## predict on test data with lambda=10
> mean((ridge.pred-y.test)^2)
[1] 168981.1
```

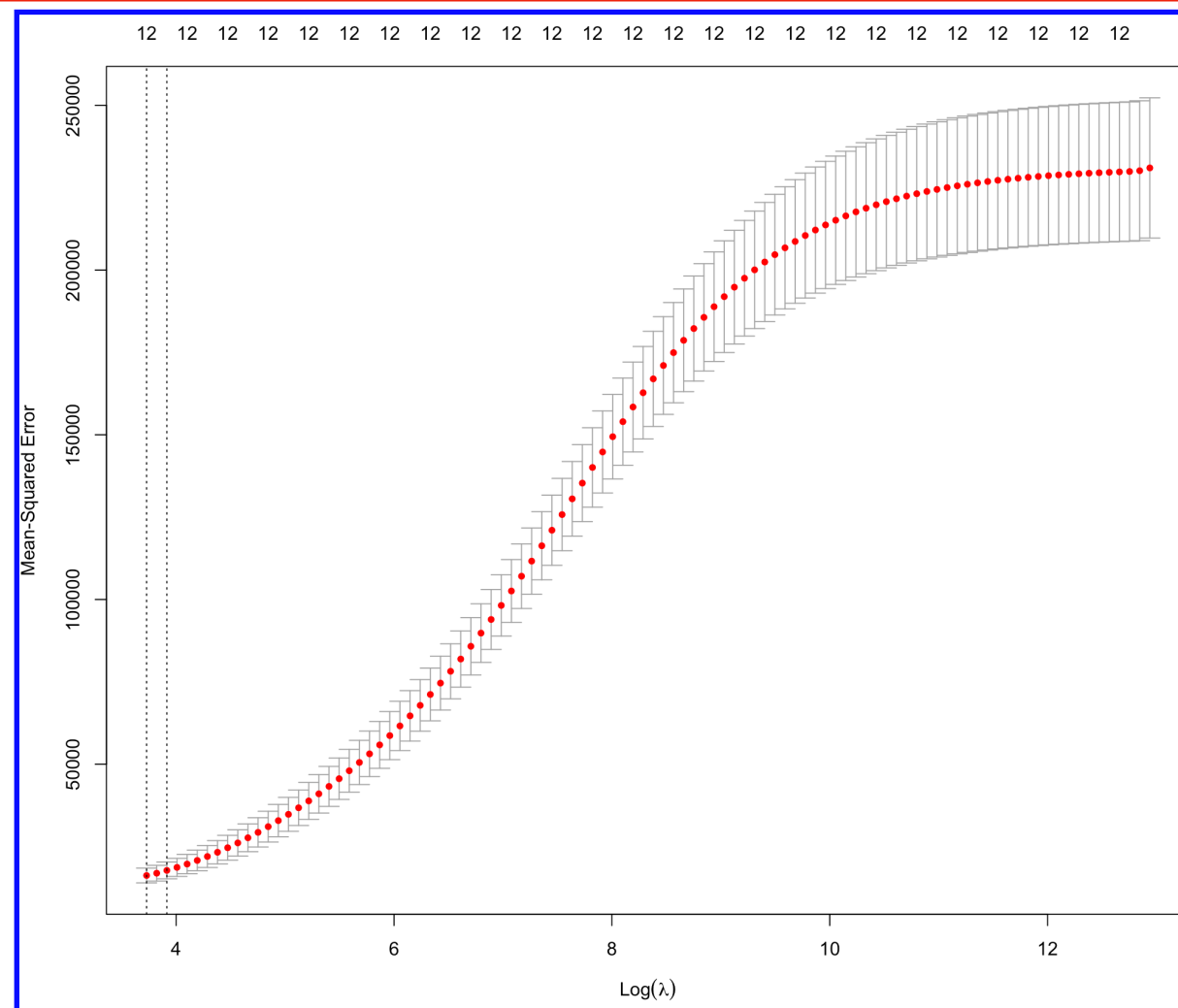
6.2 Shrinkage Methods by Regularization

Example

```
> ##### use cross validation to choose the best lambda. We can do this using the built-in cross-validation function, cv.glmnet().
> ##### By default, it implements 10-fold CV
> set.seed(1)
> cv.out=cv.glmnet(x[train,],y[train],alpha=0)    ## the default number of folds =10
> plot(cv.out)                                   ## plot CV error
> bestlam=cv.out$lambda.min                     ## which lambda returns the smallest CV error
> ridge.pred=predict(ridge.mod,s=bestlam,newx=x[test,])    ## use best lambda to predict on test data
> mean((ridge.pred-y.test)^2)
[1] 16129.68
>
>
> out=glmnet(x,y,alpha=0)
> predict(out,type="coefficients",s=bestlam)[1:13,]    ## use full dataset to fit the ridge regression model
```

```
> bestlam
[1] 41.60385
```

(Intercept)	ID	Income	Limit	Rating	Cards	Age	Education	GenderFemale	StudentYes
-402.69174822	0.02968823	-5.08362998	0.11363315	1.65055380	15.94593669	-0.97611320	-0.46045505	-3.91626143	380.23436720
MarriedYes	EthnicityAsian	EthnicityCaucasian							
-12.29347706	13.01622062	8.53524735							



6.2 Shrinkage Methods by Regularization

Bias-variance tradeoff (why ridge improves over LSE)

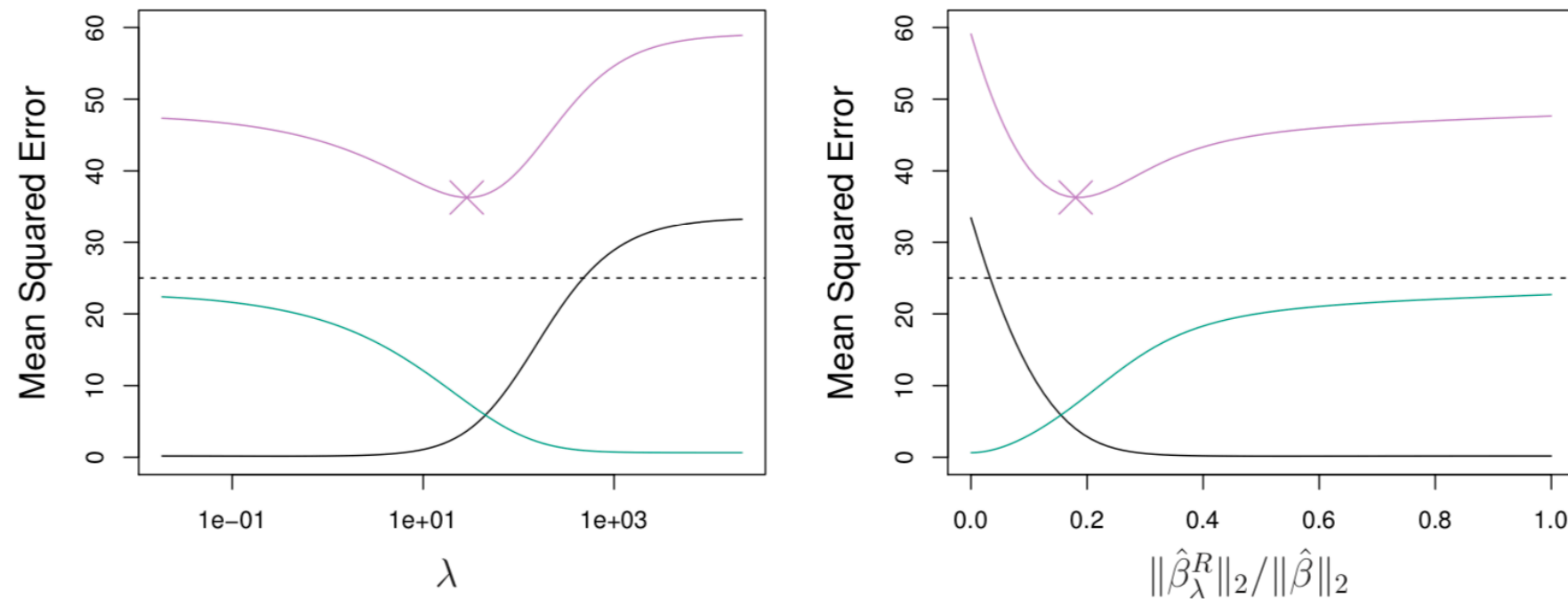


Figure: 6.5. Simulated data ($p = 45, n = 50$). Squared bias (black), variance (green), and test mean squared error (purple) for the ridge regression predictions on a simulated data set, as a function of λ and $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$. The horizontal dashed lines indicate the minimum possible MSE. The purple crosses indicate the ridge regression models for which the MSE is smallest.

6.2 Shrinkage Methods by Regularization

Remarks of Ridge regression

- ▶ *Suppose the response and the predictors is close to linear.*
- ▶ *The least squares estimates will have low bias.*
- ▶ *It may have high variance for relatively large p : small change in the training data can cause a large change in the least squares coefficient estimates.*
- ▶ *For large p , as in the example in Figure 6.5, the least squares estimates will be extremely variable.*
- ▶ *If $p > n$, then the least squares estimates do not even have a unique solution .*

6.2 Shrinkage Methods by Regularization

Remarks of Ridge regression

▶ *If $p > n$, ridge regression can still perform well by trading off a small increase in bias for a large decrease in variance.*

▶ *Ridge regression works best in situations where the least squares estimates have high variance. **Collinearity: $\mathbf{X}^T \mathbf{X}$ is close to singular***

▶ *Ridge regression also has substantial computational advantages*

$$\hat{\beta}_{\lambda}^R = (\mathbf{X}^T \mathbf{X} + \lambda I)^{-1} \mathbf{X}^T \mathbf{y}$$

▶ *where I is $p + 1$ by $p + 1$ diagonal with diagonal elements $(0, 1, 1, \dots, 1)$.*

6.2 Shrinkage Methods by Regularization

LASSO

▶ *Lasso stands for Least Absolute Shrinkage and Selection Operator.*

▶ The Lasso estimator $\hat{\beta}_\lambda^L$ is the minimizer of

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p |\beta_j|$$

▶ We may use $\|\beta\|_1 = \sum_{j=1}^p |\beta_j|$, which is the l_1 norm.

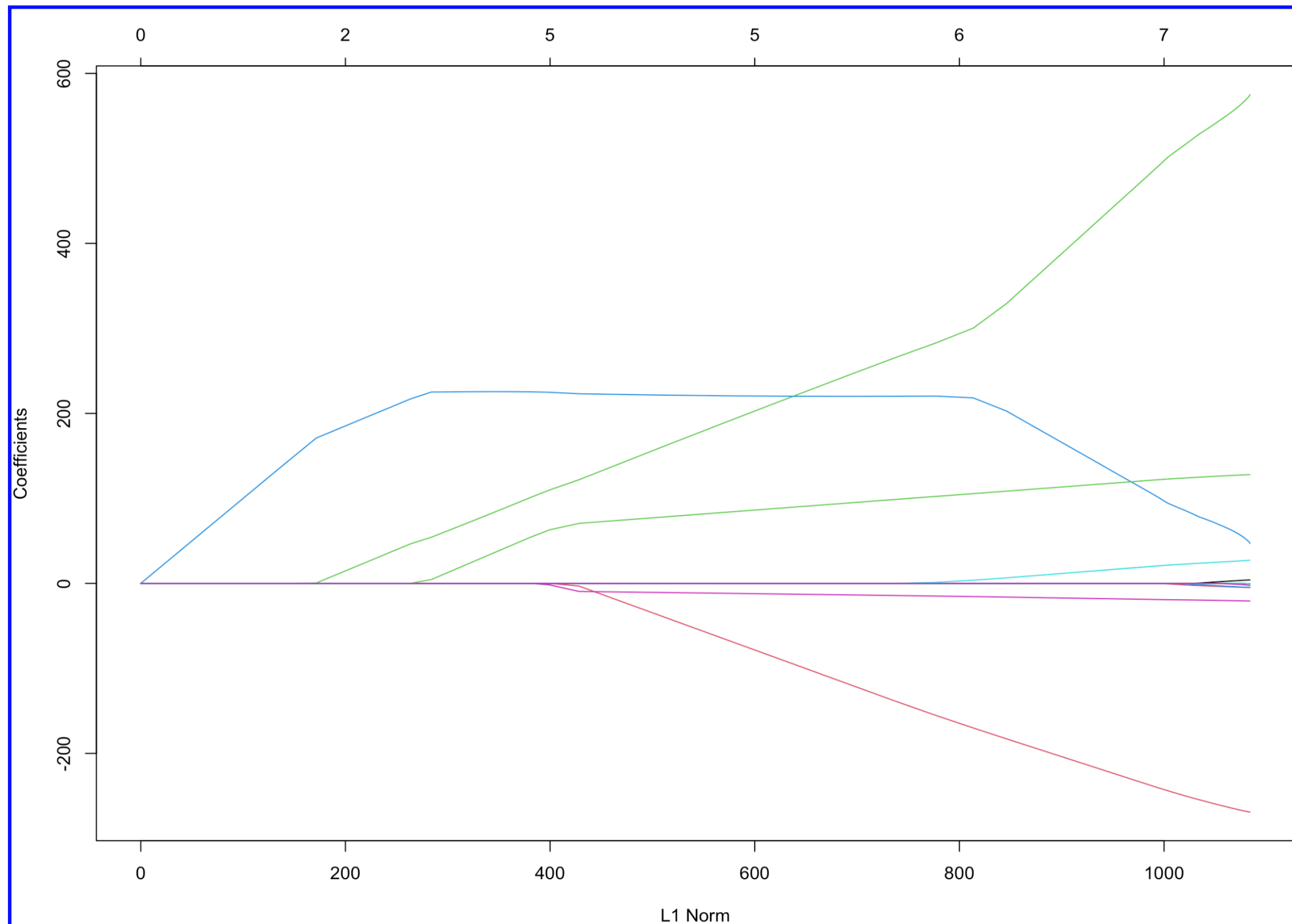
▶ *LASSO often shrinks coefficients to be identically 0. (This is not the case for ridge)*

▶ *Hence it performs variable selection, and yields sparse models.*

6.2 Shrinkage Methods by Regularization

Example: Credit data

```
##### LASSO
lasso.mod=glmnet(x[train ,],y[train],alpha=1,lambda=grid) ## fit lasso with many different lambda values
plot(lasso.mod)      ## plot the coefficients w.r.t. l1 norm
```

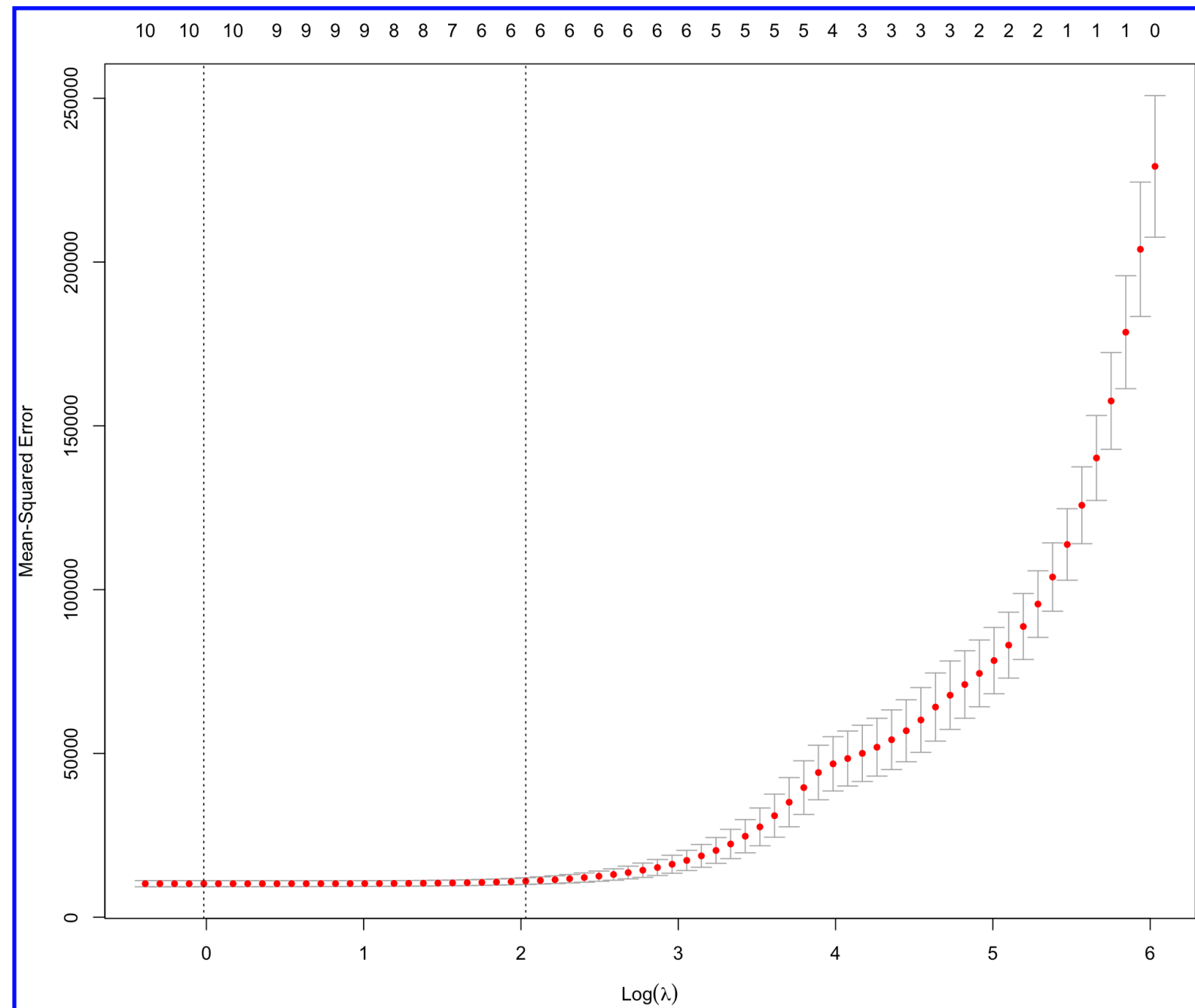


Variable Selection

6.2 Shrinkage Methods by Regularization

Example: Credit data

```
set.seed(1)
cv.out=cv.glmnet(x[train ,],y[train],alpha=1)  ## CV errors by fitting LASSO on train dataset
plot(cv.out)                                   ## plot mean squared error w.r.t. values of lambda
```



6.2 Shrinkage Methods by Regularization

Example: Credit data

```

> bestlam=cv.out$lambda.min
> bestlam
[1] 0.9837185
> lasso.pred=predict(lasso.mod,s=bestlam,newx=x[test,]) ## prediction on the test data using best lambda values
> mean((lasso.pred-y.test)^2)
[1] 10513.89
>
>
>
> ## The lasso has a substantial advantage over ridge regression in that the resulting coefficient estimates are sparse.
> out=glmnet(x,y,alpha=1,lambda=grid)
> lasso.coef=predict(out,type="coefficients",s=bestlam)[1:13,]
> lasso.coef
      (Intercept)           ID           Income           Limit           Rating           Cards           Age           Education           GenderFemale           StudentYes
520.015000      3.774317      -270.488482      399.408746      212.300690      22.073947      -10.257536      -2.276976      -3.740145      126.620309
MarriedYes EthnicityAsian EthnicityCaucasian
-3.413776      5.017349      2.363019
> lasso.coef[lasso.coef!=0]
      (Intercept)           ID           Income           Limit           Rating           Cards           Age           Education           GenderFemale           StudentYes
520.015000      3.774317      -270.488482      399.408746      212.300690      22.073947      -10.257536      -2.276976      -3.740145      126.620309
MarriedYes EthnicityAsian EthnicityCaucasian
-3.413776      5.017349      2.363019
>
>
> ## The lasso has a substantial advantage over ridge regression in that the resulting coefficient estimates are sparse. If we choose a larger regularization parameter.
> out=glmnet(x,y,alpha=1,lambda=grid)
> lasso.coef=predict(out,type="coefficients",s=20)[1:13,]
> lasso.coef
      (Intercept)           ID           Income           Limit           Rating           Cards           Age           Education           GenderFemale           StudentYes
520.015000      0.000000      -182.159322      229.341341      292.710977      1.249533      0.000000      0.000000      0.000000      104.642464
MarriedYes EthnicityAsian EthnicityCaucasian
0.000000      0.000000      0.000000
> lasso.coef[lasso.coef!=0]
      (Intercept)           Income           Limit           Rating           Cards           StudentYes
520.015000 -182.159322      229.341341      292.710977      1.249533      104.642464

```

6.2 Shrinkage Methods by Regularization

Example: Credit data

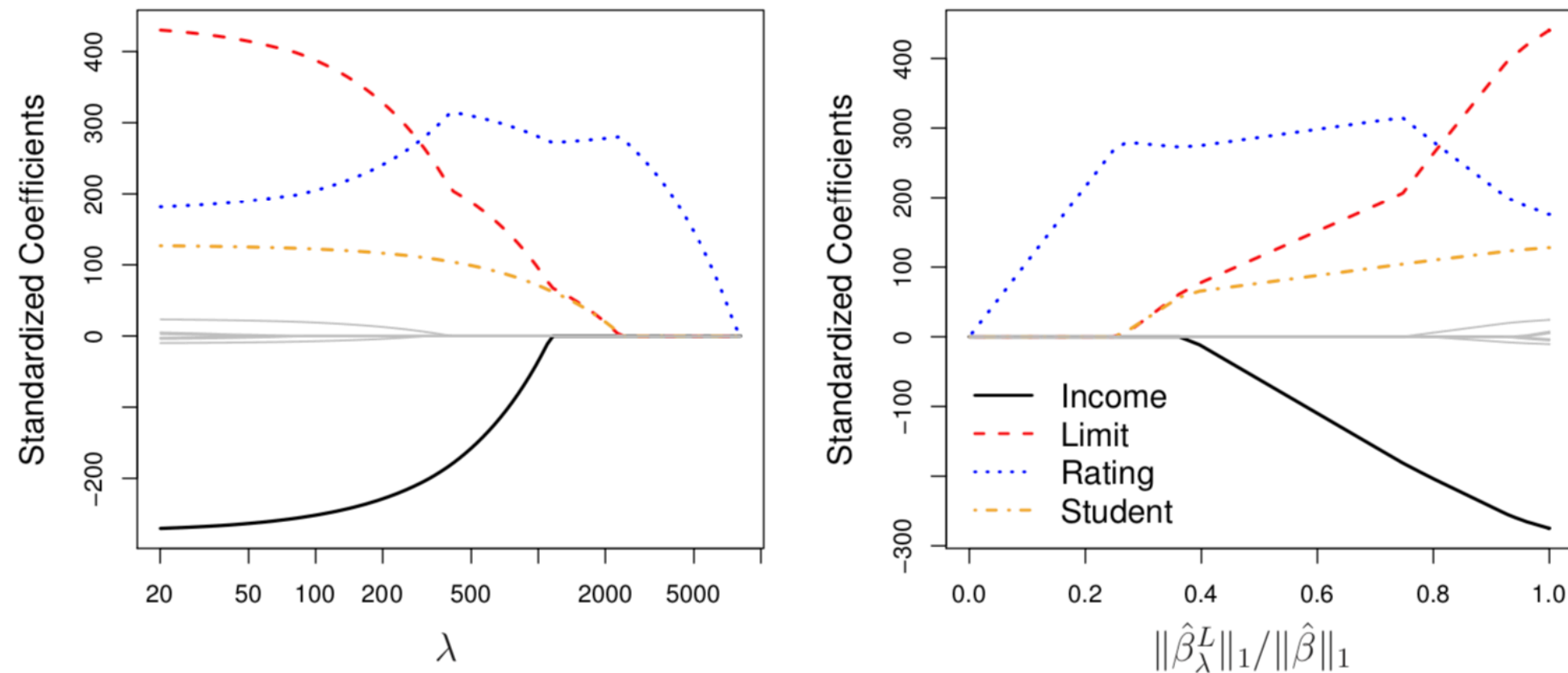


Figure: 6.6. The standardized lasso coefficients on the Credit data set are shown as a function of λ and $\|\hat{\beta}_\lambda^L\|_1 / \|\hat{\beta}\|_1$.

6.2 Shrinkage Methods by Regularization

Another formulation

For Lasso: Minimize

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 \text{ subject to } \sum_{j=1}^p |\beta_j| \leq s$$

For Ridge: Minimize

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 \text{ subject to } \sum_{j=1}^p \beta_j^2 \leq s$$

For l_0 : Minimize

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 \text{ subject to } \sum_{j=1}^p I(\beta_j \neq 0) \leq s$$

l_0 method penalizes number of non-zero coefficients. A difficult (NP-hard) problem for optimization.

6.2 Shrinkage Methods by Regularization

Equivalence between two forms

► Ridge Regression v.s. Constrained Least Squares

- Ridge Regression

$$\min_{\beta} \sum_{i=1}^n (y_i - \mathbf{x}_i^{\top} \beta)^2 + \lambda \|\beta\|^2$$

- Constrained Least Squares

$$\begin{aligned} \min_{\beta} \quad & \sum_{i=1}^n (y_i - \mathbf{x}_i^{\top} \beta)^2 \\ \text{subject to} \quad & \|\beta\|^2 \leq c \end{aligned}$$

Equivalence: For each λ , there exists a corresponding c such that solutions are equivalent, and vice versa.

► Lasso Regression v.s. Constrained ℓ_1 -Norm:

- Ridge Regression

$$\min_{\beta} \sum_{i=1}^n (y_i - \mathbf{x}_i^{\top} \beta)^2 + \lambda \|\beta\|_1$$

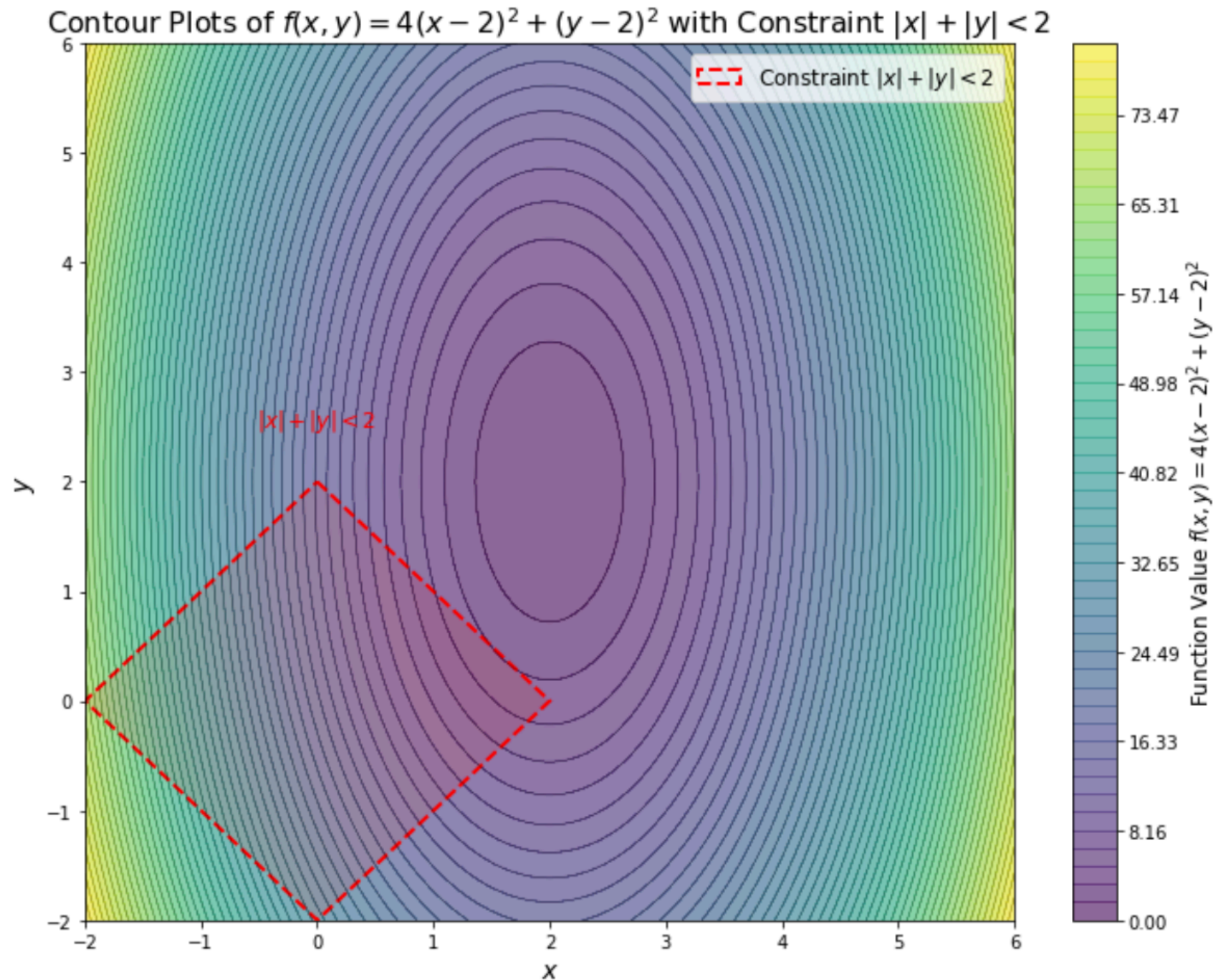
- Constrained Least Squares

$$\begin{aligned} \min_{\beta} \quad & \sum_{i=1}^n (y_i - \mathbf{x}_i^{\top} \beta)^2 \\ \text{subject to} \quad & \|\beta\|_1 \leq c \end{aligned}$$

Equivalence: For each λ , there exists a corresponding c such that solutions are equivalent, and vice versa.

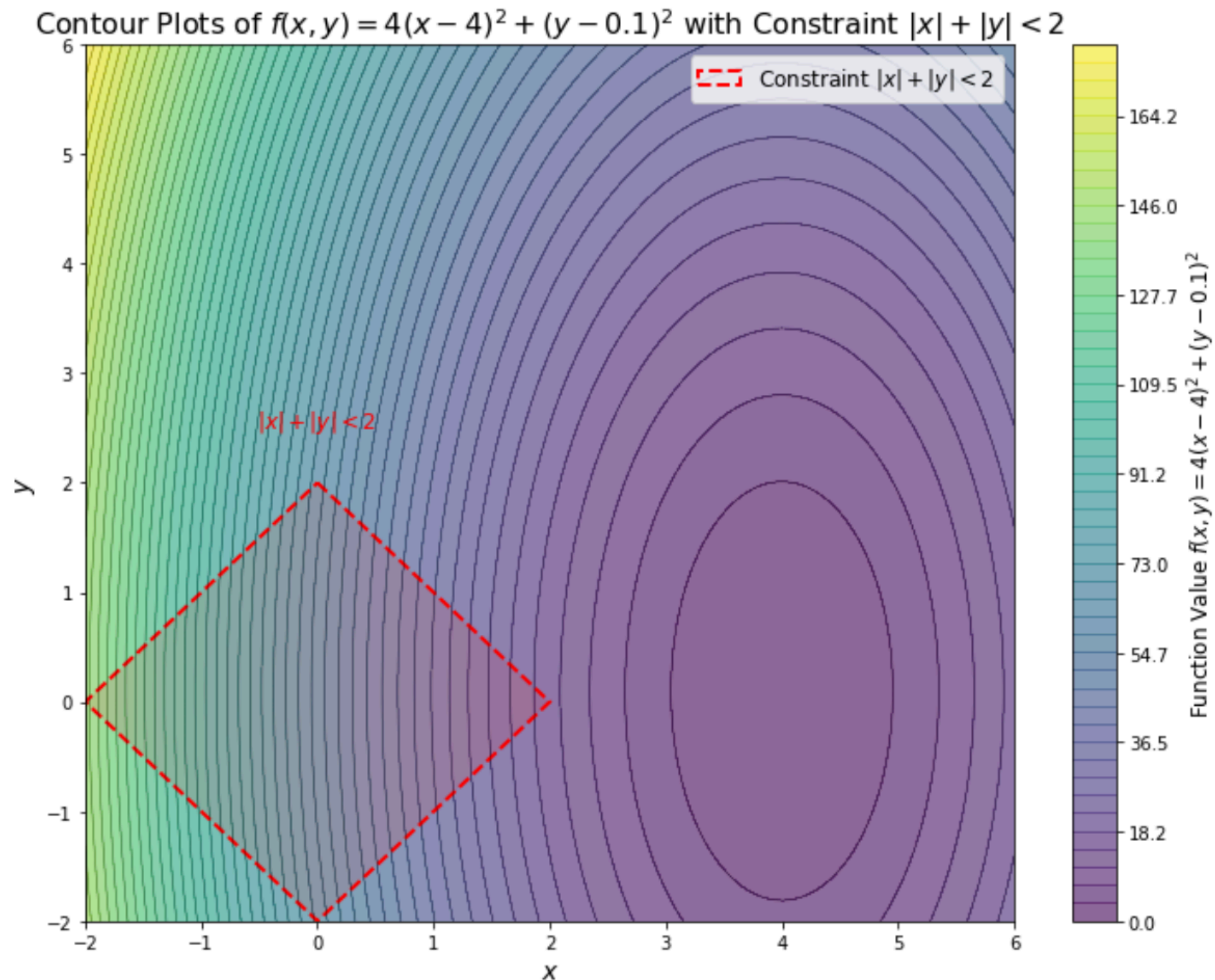
6.2 Shrinkage Methods by Regularization

Variable selection property for LASSO



6.2 Shrinkage Methods by Regularization

Variable selection property for LASSO



6.2 Shrinkage Methods by Regularization

Variable selection property for LASSO

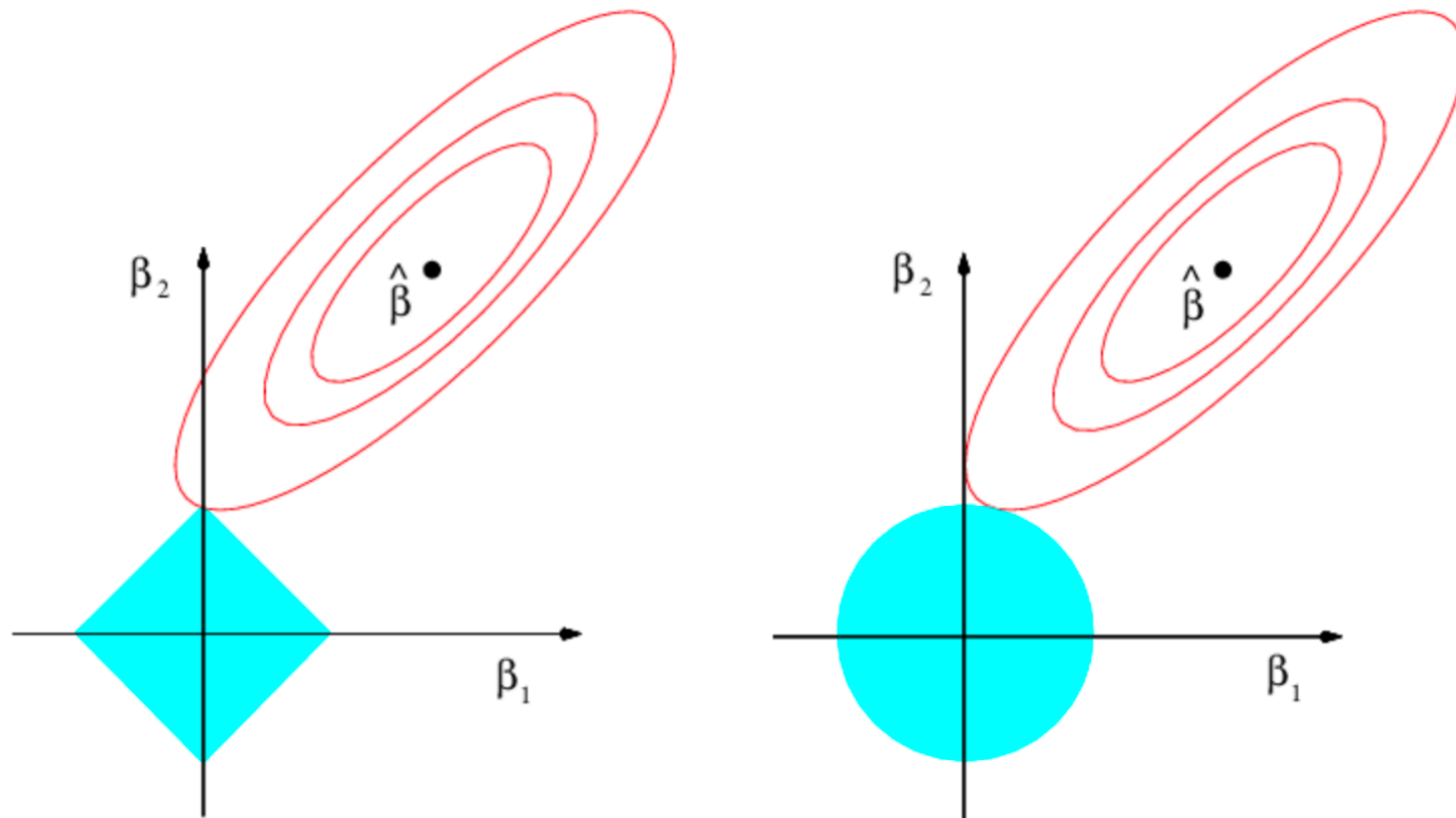


Figure: 6.7. Contours of the error and constraint functions for the lasso (left) and ridge regression (right). The solid blue areas are the constraint regions, $|\beta_1| + |\beta_2| \leq s$ and $\beta_1^2 + \beta_2^2 \leq s$, while the red ellipses are the contours of the RSS.

6.2 Shrinkage Methods by Regularization

Simulated data as in Figure 6.5 for comparing Lasso and Ridge

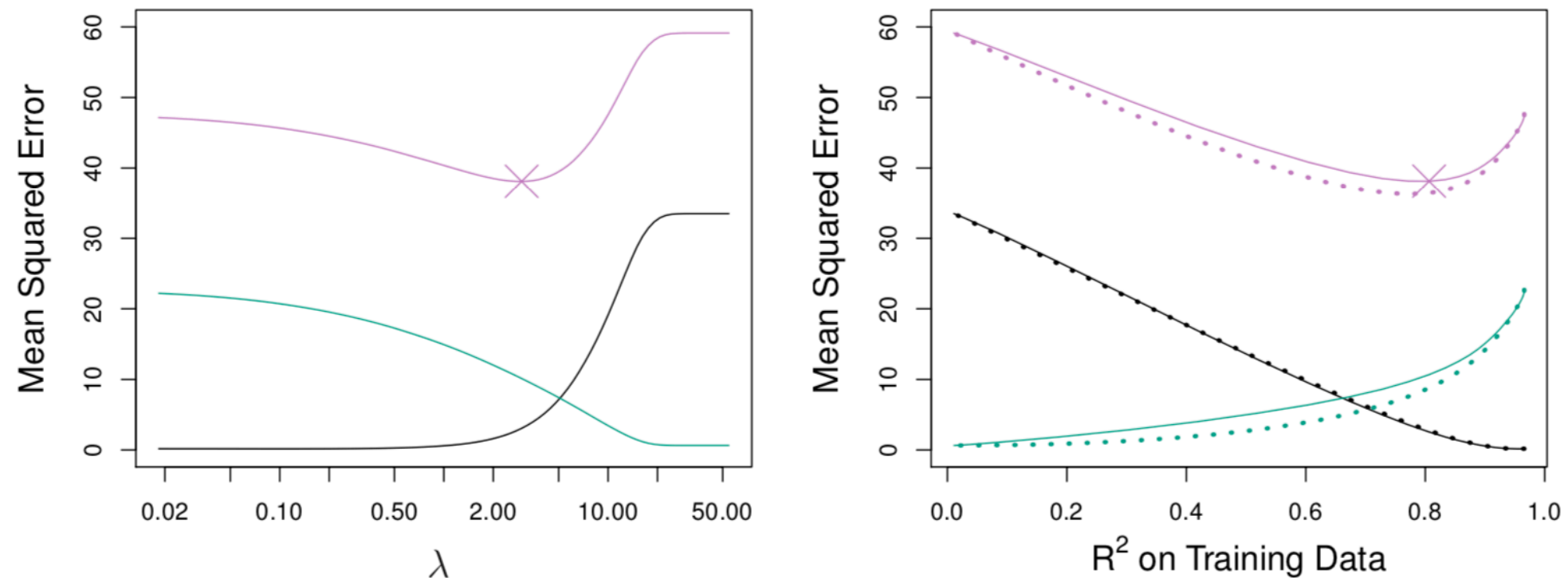


Figure: 6.8. Left: Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso on a simulated data set. Right: Comparison of squared bias, variance and test MSE between lasso (solid) and ridge (dotted). Both are plotted against their R^2 on the training data, as a common form of indexing. The crosses in both plots indicate the lasso model for which the MSE is smallest.

6.2 Shrinkage Methods by Regularization

Remarks of example on last slide

- ▶ *In the previous example, Ridge is slightly better than Lasso.*
- ▶ *The data in Figure 6.8 were generated in such a way that all 45 predictors were related to the response.*
- ▶ *None of the true coefficients $\beta_1, \dots, \beta_{45}$ equaled zero.*
- ▶ *The lasso implicitly assumes that a number of the coefficients truly equal zero.*
- ▶ *Not surprising that ridge regression outperforms the lasso in terms of prediction error in this setting.*
- ▶ *Yet lasso model is sparse, hence more interpretable than ridge.*
- ▶ *In the next figure, only 2 out of 45 predictors are related with the response.*

6.2 Shrinkage Methods by Regularization

Comparing Lasso and ridge, different data

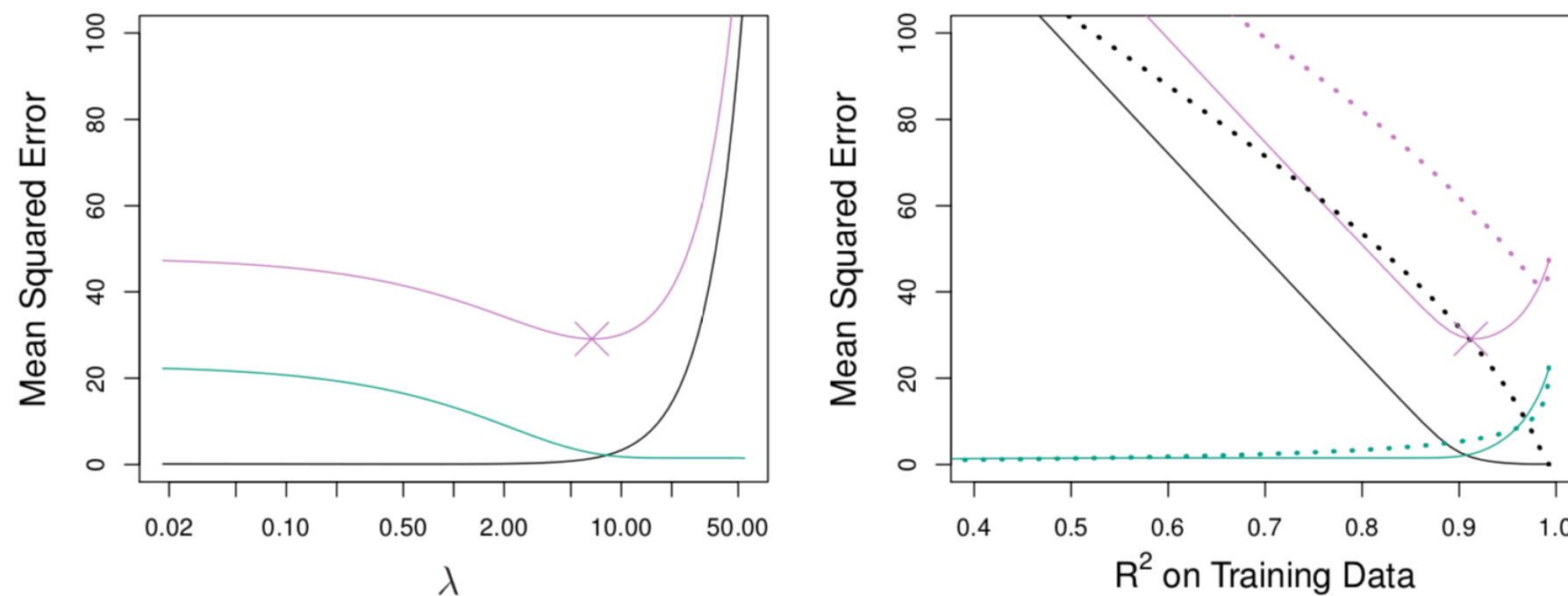


Figure: 6.9. Left: Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso. The simulated data is similar to that in Figure 6.8, except that now only two predictors are related to the response. Right: Comparison of squared bias, variance and test MSE between lasso (solid) and ridge (dotted). Both are plotted against their R^2 on the training data. The crosses in both plots indicate the lasso model for which the MSE is smallest.

6.2 Shrinkage Methods by Regularization

Summary remark

- ▶ *Both ridge and Lasso can improve over the traditional least squares by trade off variance with bias.*
- ▶ *There are significant improvement when the variance of the least squares is large, mostly with small n and large p .*
- ▶ *Lasso has feature selection, while ridge does not.*
- ▶ *Use cross validation to determine which one has better prediction.*

6.2 Shrinkage Methods by Regularization

Simple cases

▶ Ridge has closed form solution. Lasso generally does not have closed form solution.

▶ Consider the simple model $y_i = \beta_i + \epsilon_i$, $i = 1, \dots, n$ and $n = p$. Then,

The least squares $\hat{\beta}_j = y_j$; the ridge $\hat{\beta}_j^R = y_j / (1 + \lambda)$

The Lasso $\hat{\beta}_j^L = \text{sign}(y_j)(|y_j| - \lambda/2)_+$.

▶ Slightly more generally, suppose input columns of the $\mathbf{X}$ are standardized to be mean 0 and variance 1 and are orthogonal.

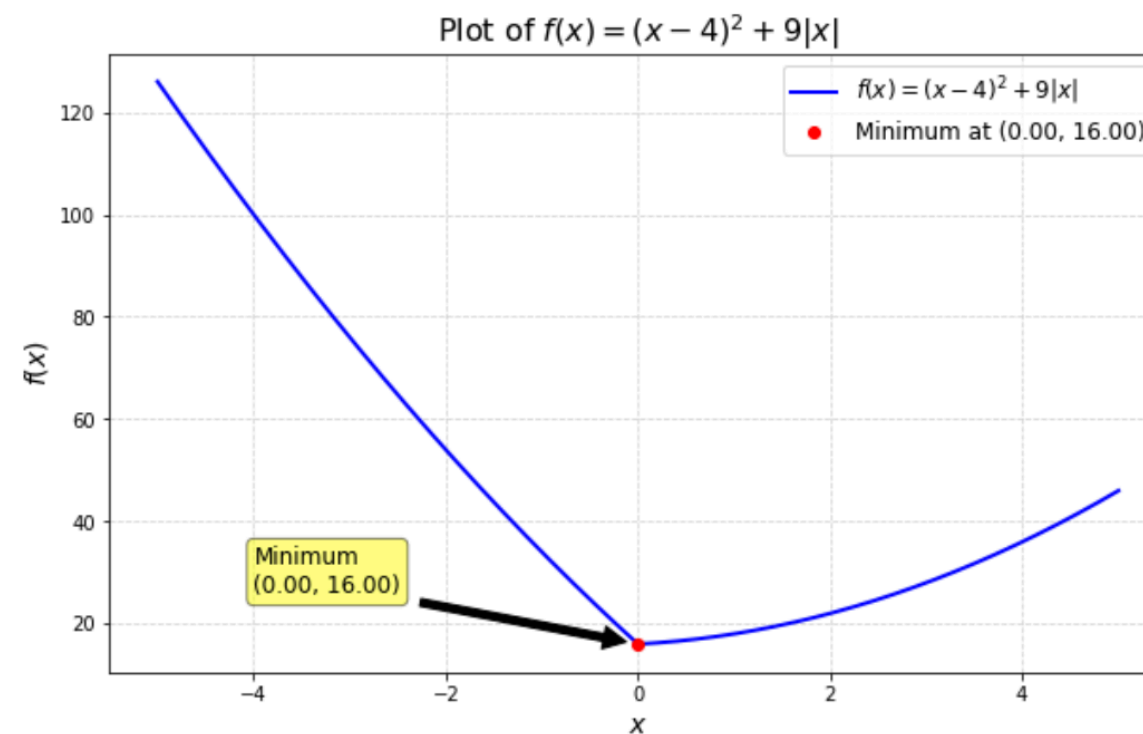
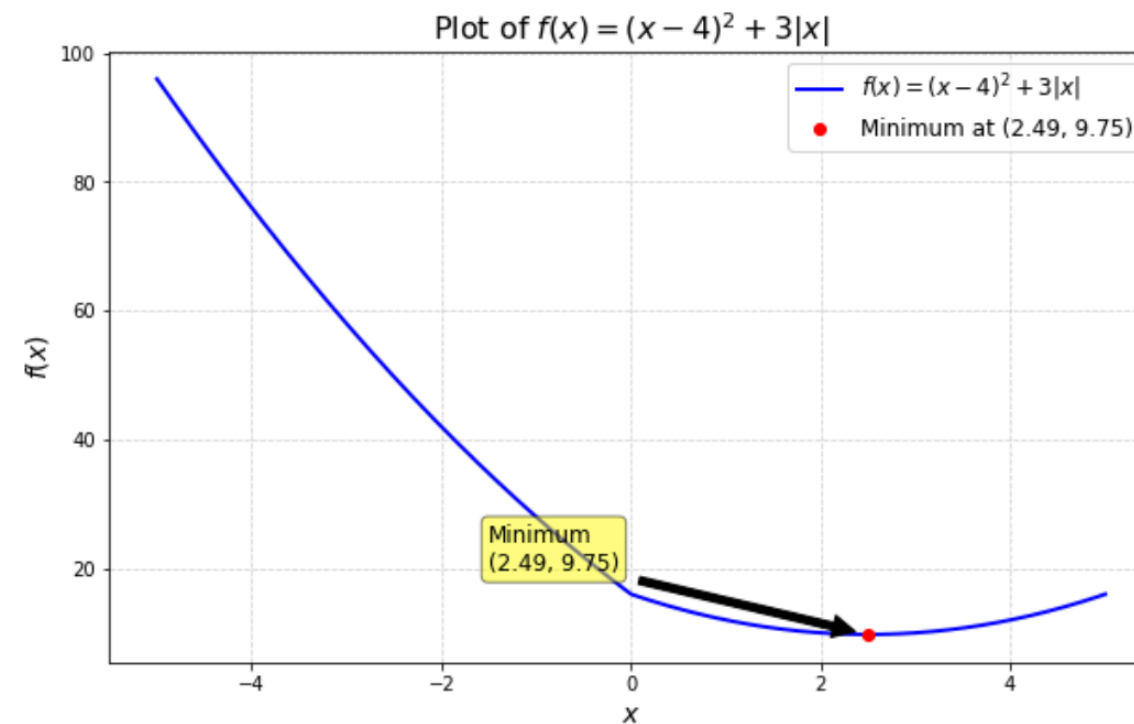
$$\hat{\beta}_j^R = \hat{\beta}_j^{\text{LSE}} / (1 + \lambda)$$

$$\hat{\beta}_j^L = \text{sign}(\hat{\beta}_j^{\text{LSE}})(|\hat{\beta}_j^{\text{LSE}}| - \lambda/2)_+$$

for $j = 1, \dots, p$.

6.2 Shrinkage Methods by Regularization

Simple cases



6.2 Shrinkage Methods by Regularization

Example

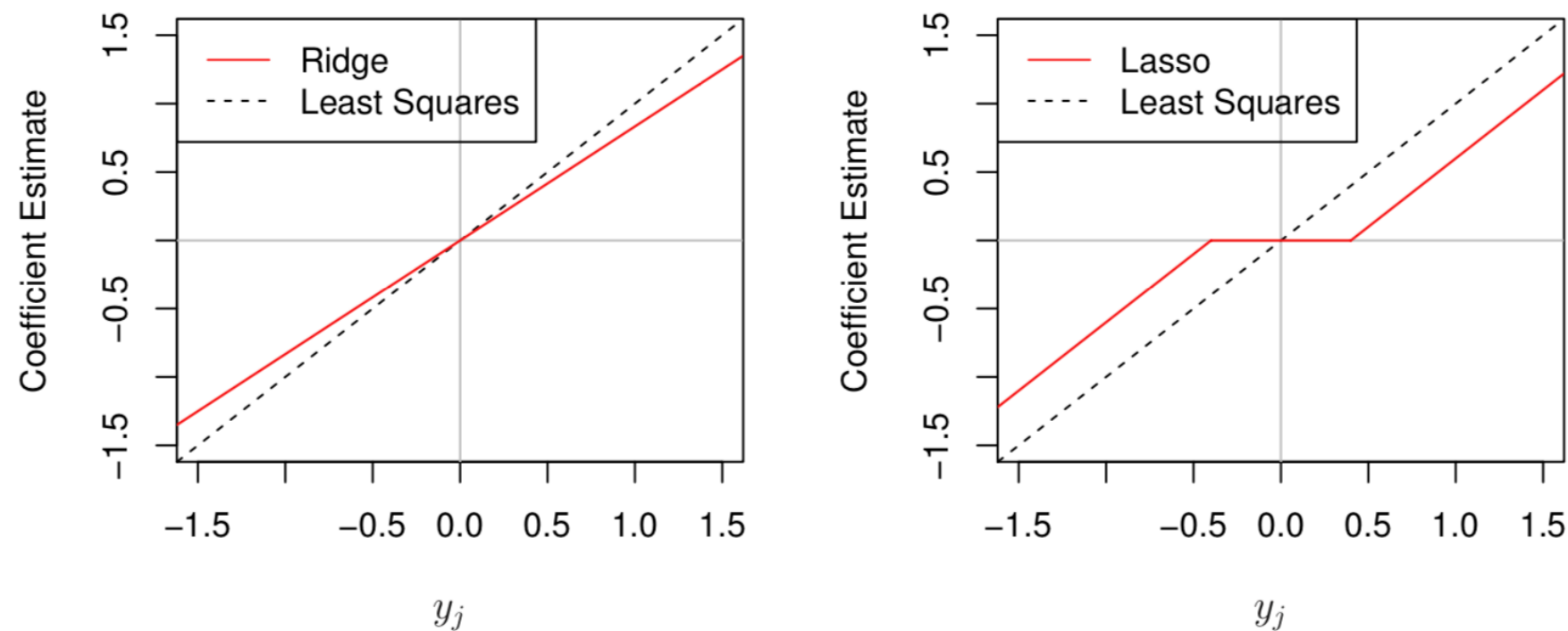


Figure: 6.10. The ridge regression and lasso coefficient estimates for a simple setting with $n = p$ and X a diagonal matrix with 1s on the diagonal. Left: The ridge regression coefficient estimates are shrunk proportionally towards zero, relative to the least squares estimates. Right: The lasso coefficient estimates are soft-thresholded towards zero.

6.2 Shrinkage Methods by Regularization

Tuning parameter selection by cross-validation: Credit data

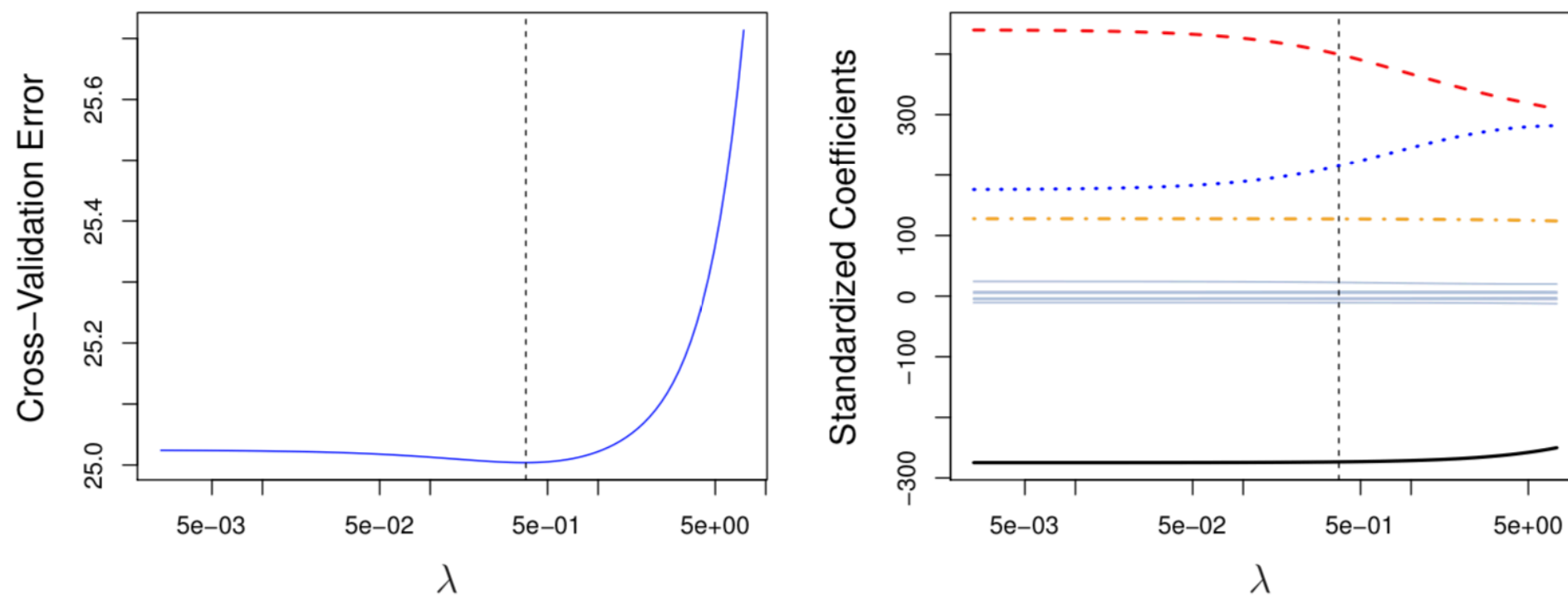


Figure: 6.12. Left: Cross-validation errors that result from applying ridge regression to the Credit data set with various value of λ . Right: The coefficient estimates as a function of λ . The vertical dashed lines indicate the value of λ selected by cross-validation.

6.2 Shrinkage Methods by Regularization

Example

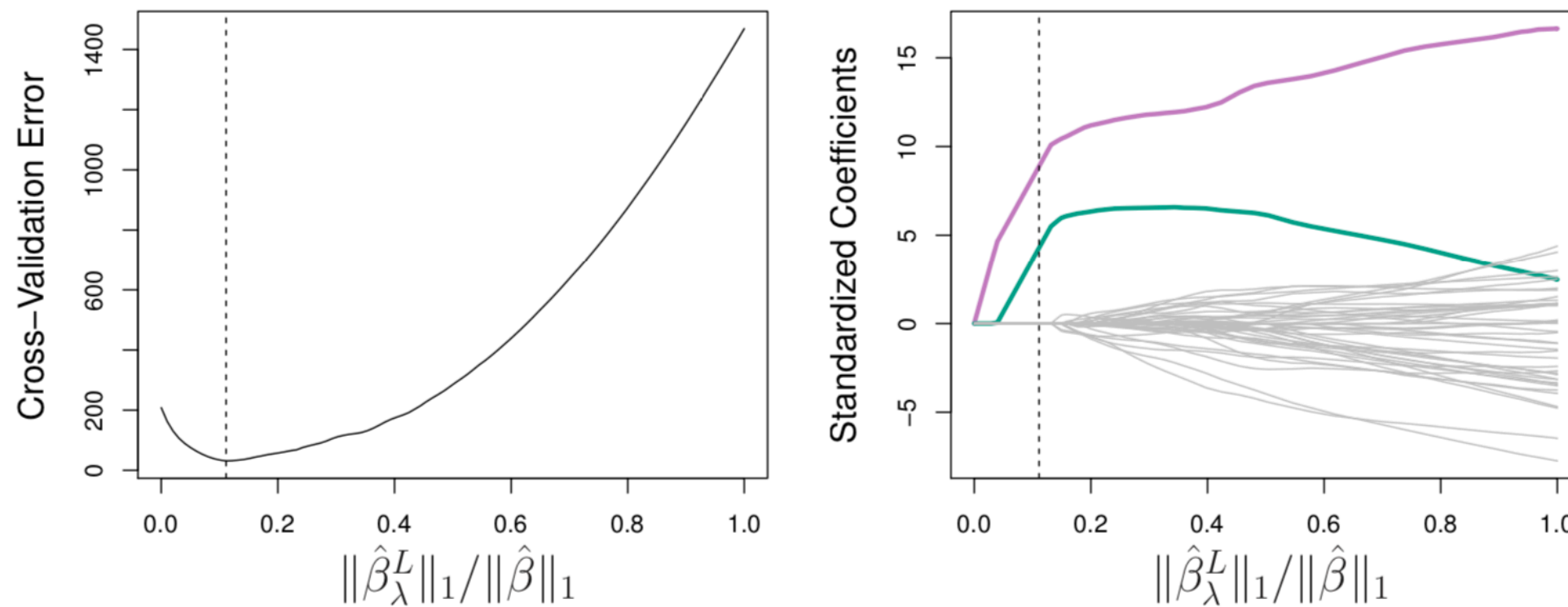


Figure: 6.13. Left: Ten-fold cross-validation MSE for the lasso, applied to the sparse simulated data set from Figure 6.9. Right: The corresponding lasso coefficient estimates are displayed. The vertical dashed lines indicate the lasso fit for which the cross-validation error is smallest.

Discussion:
how these methods be applied to non-linear model selection

- Best model selection, backward and forward selection
- Ridge regression
- LASSO

6.3 High Dimensional Data Analysis

Large p , small n

Digitization of the society brings big data.

Many of the datasets contain large number of variables.

It is common that $p \gg n$.

Example: prediction of blood pressure. **Response:** blood pressure.
Inputs: SNPs; (Individual DNA mutations).
 n may be of hundreds, but p can be of millions.

Troubles

Large p makes our linear regression model too flexible (or too large).

It can easily lead to overfit.

If $p > n$, the LSE is not even uniquely determined.

A common phenomenon: small training error, but large test error.

6.3 High Dimensional Data Analysis

Example

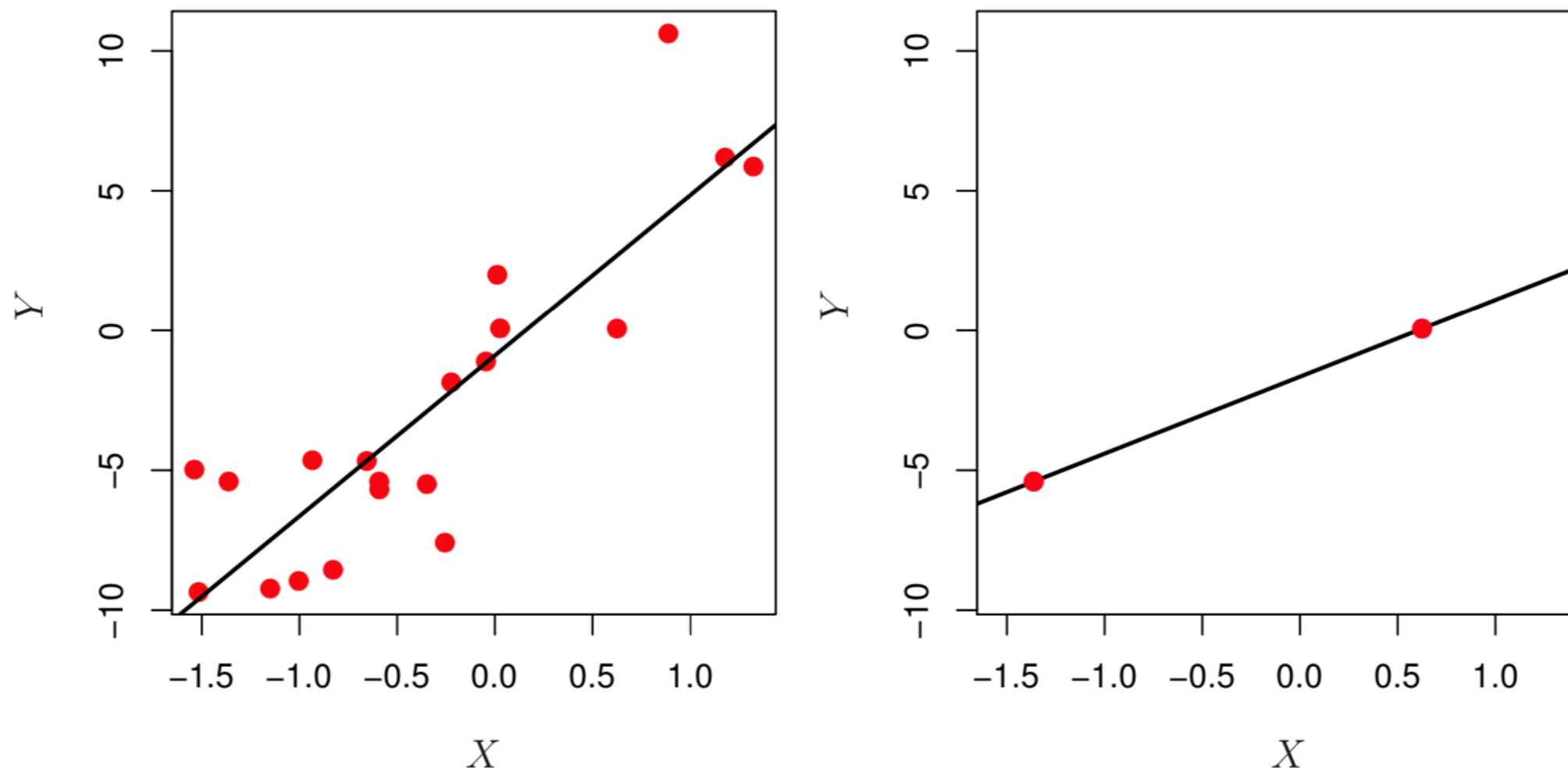


Figure: 6.22. Left: Least squares regression in the low-dimensional setting. Right: Least squares regression with $n = 2$ observations and two parameters to be estimated (an intercept and a coefficient).

6.3 High Dimensional Data Analysis

Example

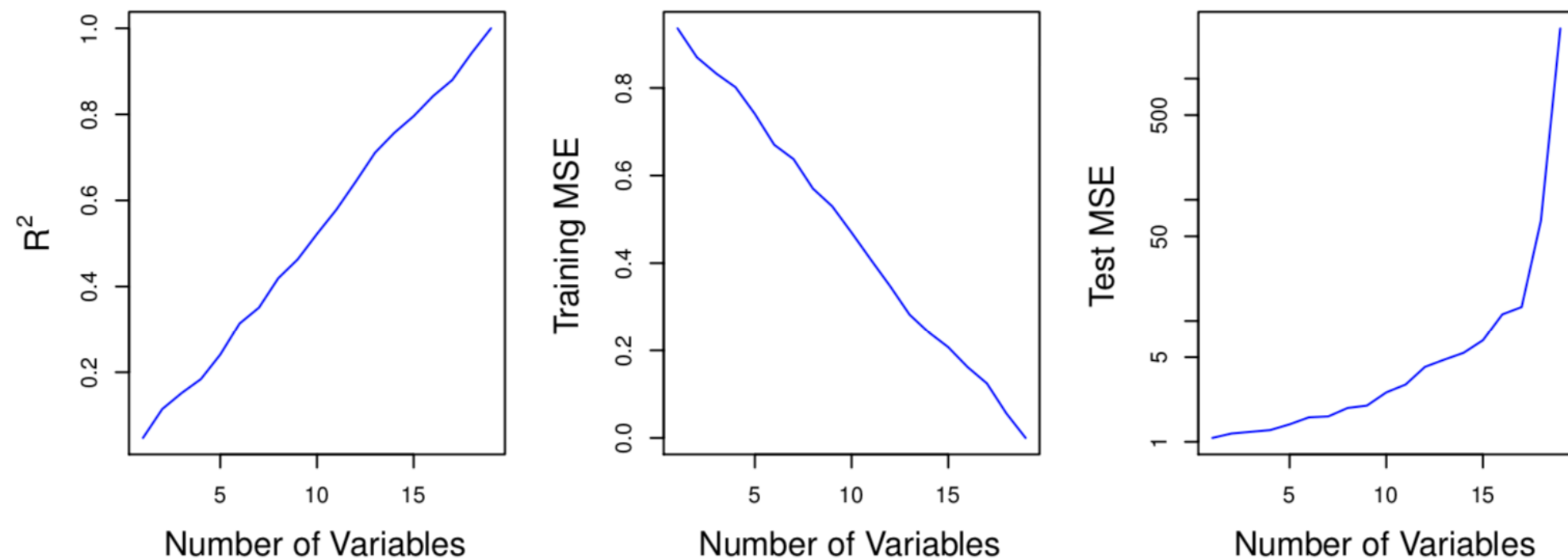


Figure: 6.23. On a simulated example with $n = 20$ training observations, features that are completely unrelated to the outcome are added to the model. Left: The R^2 increases to 1 as more features are included. Center: The training set MSE decreases to 0 as more features are included. Right: The test set MSE increases as more features are included.

6.3 High Dimensional Data Analysis

Deal with high dimensional data

- ▶ *Fit less flexible models to avoid overfit.*
- ▶ *Use forward stepwise selection, ridge regression, the lasso, and principal components regression*
- ▶ *Regularization or shrinkage plays important role.*
- ▶ *Tuning parameter choice*

6.3 High Dimensional Data Analysis

Example for curse of dimensionality

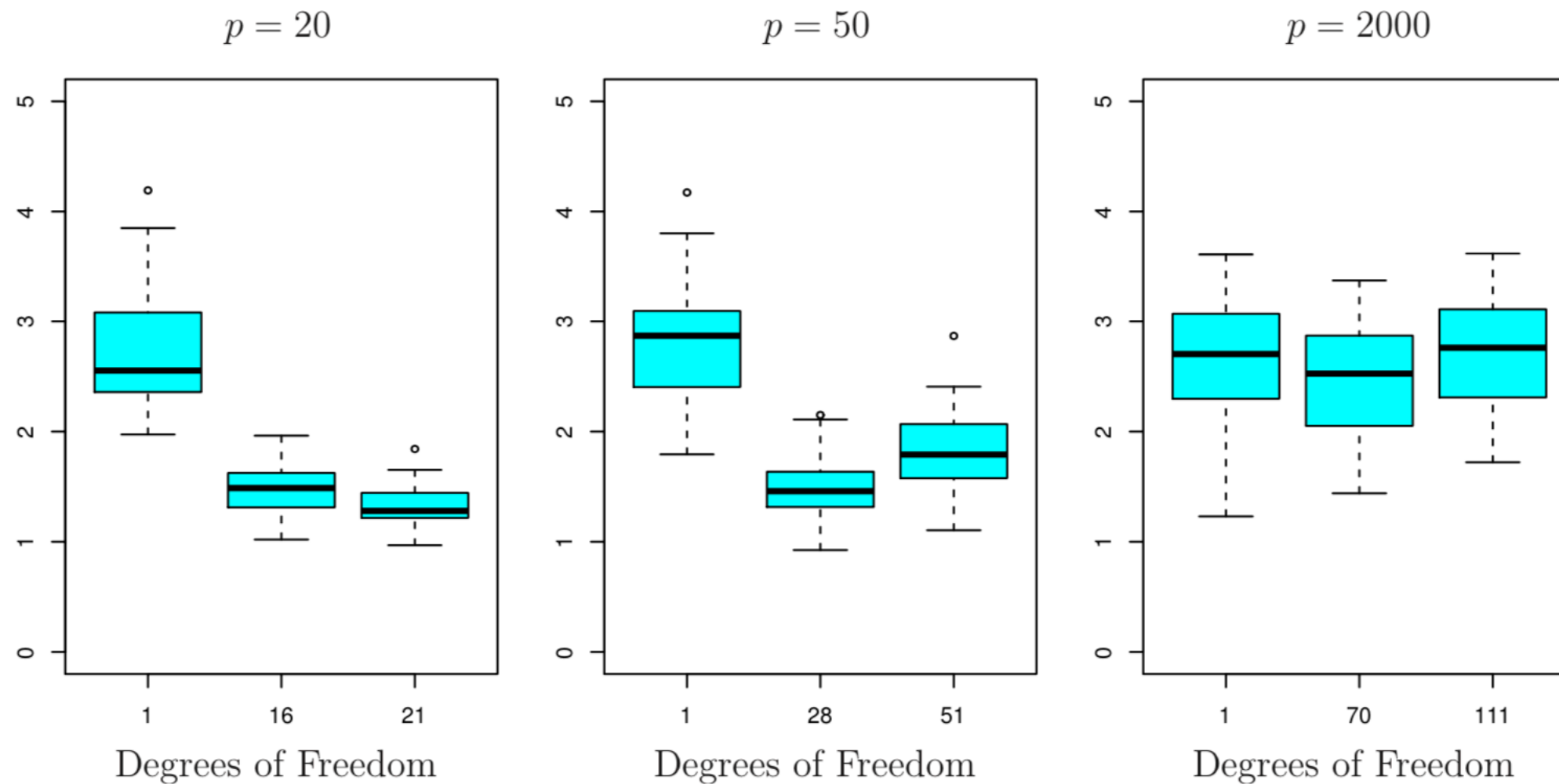


Figure: 6.24. see next page

6.3 High Dimensional Data Analysis

Example for curse of dimensionality

Figure 6.24. The lasso was performed with $n = 100$ observations and three values of p , the number of features. Of the p features, 20 were associated with the response. The boxplots show the test MSEs that result using three different values of the tuning parameter λ in (6.7). For ease of interpretation, rather than reporting λ , the degrees of freedom are reported; for the lasso this turns out to be simply the number of estimated non-zero coefficients. When $p = 20$, the lowest test MSE was obtained with the smallest amount of regularization. When $p = 50$, the lowest test MSE was achieved when there is a substantial amount of regularization. When $p = 2,000$ the lasso performed poorly regardless of the amount of regularization, due to the fact that only 20 of the 2,000 features truly are associated with the outcome.

6.3 High Dimensional Data Analysis

Caution when $p > n$

- ▶ *Extreme multicollinearity.*
- ▶ *Refrain from over-statement. (What we find may be one of many possible models.)*
- ▶ *Avoid using sum of squares, p -values, R^2 , or other traditional measures of model on training as evidence of good fit.*
- ▶ *Place more emphasis on test error or cross validation error.*